Department of Computer Science
&
Engineering

PROJECT FILE

TITLE OF PROJECT:

QuantRegime AI: Markov Chain & Machine Learning Asset Forecaster



FACULTY GUIDE:
Dr. Seema

STUDENT:
Mrigank(2534047)
Nikhil (2534068)
Mohit Siwach (2534044)
3CSE D2
Branch: Computer Science & Engineering

Chandigarh Group of Colleges
College of Engineering
Landran, Mohali-140307

CERTIFICATE

I hereby certify that the work which is being presented in the B.Tech. Major Project Report entitled **"QuantRegime AI: Markov Chain & Machine Learning Asset Forecaster"**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology in Computer Science & Engineering** and submitted to the Department of Computer Science & Engineering of CGC- College of Engineering Landran, Mohali Punjab is an authentic record of my own work carried out during a period from 2025-26 (**3rd semester**) under the supervision of Dr. Seema, **Department of CSE**.

The matter presented in this Project Report has not been submitted by me for the award of any other degree elsewhere.

Signature of Student
Mrigank(2534047)
Nikhil (2534068)
Mohit Siwach (2534044)

This is to certify that the above statement made by the student is correct to the best of my knowledge.

Signature of Supervisor

Date:

Dr. Seema

Head of Department of Computer Science & Engineering
Dr. Sushil Kamboj

ACKNOWLEDGEMENT

I would like to place on record my deep sense of gratitude to Prof. Sushil Kamboj, HOD-Dept. **Computer Science & Engineering**, CGC- College of Engineering Landran, Mohali, India for his generous guidance, help and useful suggestions.

I express my sincere gratitude to Dr. Seema, Dept. **Computer Science & Engineering**, CGC-College of Engineering Landran, Mohali, India, for his stimulating guidance, continuous encouragement and supervision throughout the course of present work.

I also wish to extend my thanks to faculty and other colleagues for attending my seminars and for their insightful comments and constructive suggestions to improve the quality of this project work.

I am extremely thankful to Anil Kumar Gupta, Principal, CGC- College of Engineering Landran, Mohali, for providing me infrastructural facilities to work in, without which this work would not have been possible.

Signature of Student
Mrigank(2534047)
Nikhil (2534068)
Mohit Siwach (2534044)

ABSTRACT

Financial markets alternate between phases of steady growth, consolidation and turbulent decline, so the relation between past prices and future returns is not constant over time. A single predictive model fitted over a long history averages across these phases. This project presents QuantRegime AI, a Python system that detects market regimes, models the transitions between them, trains regime-specific regression models and uses all three components to simulate future price paths and quantify downside risk.

Three regimes (Bull, Neutral and Bear) are discovered without labels by a Gaussian Mixture Model on standardised daily return, 20-day realised volatility and moving-average trend spread. A first-order discrete-time Markov chain is fitted to the detected regime sequence to obtain the transition matrix, the long-run stationary distribution and the expected holding time of each regime. Ridge, polynomial and random-forest regressors are then trained separately for each regime and benchmarked against a global model. A Monte Carlo engine chains regime transitions with regime-specific drift and resampled residual shocks to generate forward price paths, the 50% and 80% probability bands and the 95% Value-at-Risk. The whole pipeline is delivered as an interactive Streamlit dashboard with stress-test controls, an offline synthetic-data fallback and a 300-DPI figure export. The implementation comprises about 1,680 lines of Python.

The system was evaluated on a seeded synthetic series of 750 business days whose hidden regimes are known. The detected regimes are interpretable and persistent, with self-transition probabilities of 0.931, 0.897, 0.798 and expected holding times of 14.5, 9.7, 5.0 days, but they agree with the generator's hidden states on only 40% of days because the features average over longer windows than the generator's regimes last. Regime conditioning improves in-sample fit and, over 30 independent series, raises directional accuracy by 1.8 percentage points on the fitting data; on a chronological hold-out, however, the mean difference is -0.4 percentage points ($p = 0.63$) and the hold-out R^2 is negative for both predictors. The improvement is therefore an in-sample effect.

In the base simulation of 150 paths over 30 trading days the 95% Value-at-Risk is -9.8%. Forcing a five-day Bear regime roughly doubles it to -20.2%, and scaling the shocks scales the tail almost proportionally. The Value-at-Risk estimate is noisy at 150 paths, and the direction of the central forecast depends on the regressor chosen. Testing also revealed two software defects (a weekend-dependent failure of the synthetic-data generator under pandas 3 and a sign-dependent failure of the stationary distribution) together with several mismatches between claims and code; all were corrected and the corrected version was re-tested.

It is concluded that QuantRegime AI is a transparent framework for regime description and scenario-based risk exploration, but that the evidence does not support the claim that regime conditioning improves forecasting skill. Walk-forward evaluation on real data, model-based regime estimation and consistent feature updating in the simulator are proposed as future work.

Keywords: market regimes; Gaussian Mixture Model; Markov chain; ridge regression; Monte Carlo simulation; Value-at-Risk; Streamlit.

TABLE OF CONTENTS

Page No

Chapter 1: INTRODUCTION....1
1.1 General....1
1.2 Background....1
1.2.1 Financial time series and their stylised facts....1
1.2.2 Market regimes....2
1.2.3 Markov chains....2
1.2.4 Machine learning regression....2
1.2.5 Monte Carlo simulation and Value-at-Risk....3
1.3 Motivation....3
1.4 Overview of the Proposed System....3
1.5 Scope and Limitations....5
1.6 Organisation of the Report....5

Chapter 2: LITERATURE SURVEY....6
2.1 Stylised Facts of Financial Returns....6
2.2 Regime-Switching Models....6
2.3 Markov Chain Theory....7
2.4 Unsupervised Clustering for Regime Identification....7
2.5 Machine Learning for Return Prediction....7
2.6 Out-of-Sample Evidence and Over-fitting....8
2.7 Monte Carlo Simulation and Risk Measurement....8
2.8 Software Ecosystem....8
2.9 Summary of the Survey and Research Gap....9

Chapter 3: PROBLEM STATEMENT AND OBJECTIVES....10
3.1 Problem Statement....10
3.2 Objectives....11
3.3 Mathematical Formulation....12
3.3.1 Returns and features....12
3.3.2 Gaussian Mixture Model....12
3.3.3 Discrete-time Markov chain....13
3.3.4 Regime-conditioned regression....13
3.3.5 Monte Carlo simulation and Value-at-Risk....14
3.3.6 Evaluation metrics....14
3.4 Assumptions and Constraints....15

Chapter 4: IMPLEMENTATION METHODOLOGY....16
4.1 System Architecture....16
4.2 Technology Stack....17
4.3 Data Acquisition....17
4.3.1 Live market data....17
4.3.2 Synthetic regime-switching generator....18
4.4 Feature Engineering....18
4.5 Regime Detection Module....19
4.6 Markov Chain Module....19
4.7 Regime-Conditioned Regression Module....20
4.8 Monte Carlo Simulation Module....20

4.9 Dashboard and Visualisation Layer....22
4.10 Verification and Experimental Design....23
4.11 Deployment....24

Chapter 5: RESULTS AND DISCUSSION....25
5.1 Dataset Summary....25
5.2 Regime Detection Results....25
5.2.1 Detected regimes....25
5.2.2 Agreement with the generating regimes....27
5.3 Markov Chain Results....28
5.4 Regime-Conditioned Regression Results....31
5.4.1 In-sample comparison....31
5.4.2 Chronological hold-out....32
5.4.3 Robustness over 30 synthetic series....33
5.5 Monte Carlo Simulation Results....34
5.5.1 Base forecast....34
5.5.2 Stress scenarios....36
5.5.3 Sampling error and sensitivity to the model....37
5.5.4 Exported academic figure....38
5.6 Testing Results and Issues Found....39
5.7 Summary of Findings....41

Chapter 6: CONCLUSION AND FUTURE SCOPE....42
6.1 Conclusion....42
6.2 Contributions....42
6.3 Limitations....43
6.4 Future Scope....43

References....45

Appendix A: MODULE AND INTERFACE REFERENCE....48

Appendix B: INSTALLATION AND REPRODUCTION....49
B.1 Running the application....49
B.2 Reproducing the experiments....49

Appendix C: RESULTS FOR THE 30 SYNTHETIC SERIES....50

Appendix D: SOURCE CODE....51
D.1 requirements.txt....51
D.2 run.sh....51
D.3 test_pipeline.py....51
D.4 src/data_loader.py....52
D.5 src/regime_detector.py....53
D.6 src/markov_model.py....55
D.7 src/ml_regressor.py....57
D.8 src/monte_carlo.py....59
D.9 src/visualizations.py....60
D.10 app.py....67

List of Tables

Table	Title	Page
2.1	Summary of the surveyed literature and its use in this project	9
3.1	Project objectives and their measurable outcomes	12
4.1	Repository modules, size and responsibility	16
4.2	Technology stack	17
4.3	Parameters of the synthetic data generator	18
4.4	Engineered features	19
4.5	Regressor configurations	20
4.6	Sidebar controls of the dashboard	22
4.7	Analysis tabs of the dashboard	23
4.8	Experiments run for the evaluation	24
5.1	Summary statistics of the synthetic series (seed 42)	25
5.2	Regime statistics produced by the dashboard	26
5.3	Mean feature values by detected regime	26
5.4	Generator regimes (rows) versus detected regimes (columns), number of days	27
5.5	Spell lengths of the generator and detected regimes (days)	28
5.6	Observed transition counts between detected regimes	29
5.7	Stationary distribution and expected holding time	30
5.8	In-sample comparison of global and regime-conditioned predictors	31
5.9	Hold-out comparison (training rows 1 to 490, test rows 491 to 700)	32
5.10	Hold-out metrics as displayed by the corrected dashboard (GMM labels from the full history)	33
5.11	Ridge model over 30 independent synthetic series	33
5.12	Base-run Monte Carlo output	35
5.13	Monte Carlo results under stress scenarios (150 paths, 30 days)	36
5.14	Effect of the number of simulated paths (seed 2026)	37
5.15	Sensitivity of the Monte Carlo output to the regressor (100 paths)	38
5.16	Results of the repository test script	39
5.17	Corrections applied to the repository	39
5.18	Verification of the corrected version	40
5.19	Answers to the research questions	41
A.1	Public classes and functions of the repository	48
C.1	Per-seed results for the ridge model (DA in %, hold-out = last 30% of rows)	50

List of Figures

Figure	Title	Page
1.1	Processing chain of the QuantRegime AI system	4
3.1	Simulated price, daily log-returns and 20-day realised volatility, showing alternating calm and turbulent periods	11
4.1	Layered architecture of QuantRegime AI	16
4.2	Flow chart of one simulated day	21
5.1	Price shaded by detected regime (a) and detected versus generator regime sequence (b)	26
5.2	Detected regimes in feature space: return versus volatility (a) and trend spread versus volatility (b)	27
5.3	Confusion matrix of detected versus generator regimes	28
5.4	Estimated transition matrix (a) and generator transition matrix (b)	29
5.5	Markov state-transition network of the detected regimes	29
5.6	Stationary distribution (a) and expected holding time (b), estimated versus generator	30
5.7	In-sample R^2 (a) and directional accuracy (b) of the global and regime-conditioned models	31
5.8	Gain of regime conditioning over the global model across 30 series, in-sample and on hold-out	33
5.9	Monte Carlo forecast fan: 150 paths, 30 trading days	35
5.10	Distribution of simulated 30-day returns with median and 95% VaR	35
5.11	Median simulated price under five scenarios with P10–P90 bands for the natural and Black Swan runs	36
5.12	95% VaR as a function of the number of simulated paths	37
5.13	Four-panel figure exported by generate_matplotlib_report_figure for the synthetic series	38

NOMENCLATURE

English Symbols

P_t	Closing price on trading day t
r_t	Daily log-return, $\ln(P_t/P_{t-1})$
$\mathbf{x}_t$	Feature vector of day t (five indicators)
S_t	Market regime on day t (0 Bull, 1 Neutral, 2 Bear)
$\mathbf{P}$, p_{ij}	Transition probability matrix and its entry for the move from state i to j
N_{ij}	Number of observed transitions from state i to state j
$E[T_i]$	Expected holding time of state i, $1/(1 - p_{ii})$
f_s	Regression function trained on the days of regime s
y_t	Target: next-day log-return
M_t	10-day momentum
K	Number of mixture components (3)
N	Number of simulated paths
R_τ	Simulated horizon return
VaR_{95}	95% Value-at-Risk
n	Number of evaluation observations

Greek Symbols

α	Ridge regularisation strength
β	Regression coefficient vector
γ_{tk}	Responsibility of component k for day t
Δ_t	Trend spread, SMA20/SMA50 − 1
ε	Residual shock
λ	Volatility stress multiplier
μ_k	Mean vector of mixture component k
$\boldsymbol{\pi}$	Stationary (equilibrium) distribution
σ_t	20-day annualised realised volatility
Σ_k	Covariance matrix of mixture component k
τ	Forecast horizon in trading days
ω_k	Mixture weight of component k

ABBREVIATIONS/ACRONYMS

ARCH / GARCH	Autoregressive Conditional Heteroscedasticity / Generalised ARCH
ARI	Adjusted Rand Index
CGC	Chandigarh Group of Colleges
CSE	Computer Science & Engineering
DA	Directional Accuracy
DPI	Dots Per Inch
DTMC	Discrete-Time Markov Chain
EM	Expectation–Maximisation
GBM	Geometric Brownian Motion
GMM	Gaussian Mixture Model
HMM	Hidden Markov Model
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
NMI	Normalised Mutual Information
OHLCV	Open, High, Low, Close, Volume
RMSE	Root Mean Squared Error
RSI	Relative Strength Index
SMA	Simple Moving Average
VaR	Value-at-Risk

CHAPTER 1: INTRODUCTION

1.1 General

Financial markets produce a continuous stream of price observations that investors, risk managers and researchers use to judge what may happen next. Forecasting these prices is difficult because the underlying process is noisy, adaptive and only weakly predictable. A practical forecasting system therefore has to do more than output a single number. It has to describe the conditions under which the number was produced, quantify the uncertainty around it and make its reasoning visible to the person using it.

One of the most important sources of difficulty is that the statistical behaviour of a market does not stay constant. Long periods of steady appreciation with low volatility are followed by sideways consolidation and then by sharp, volatile declines. Practitioners describe these phases as *market regimes* [24]. A single predictive model that is estimated over a long history is forced to average across all of these phases, and the resulting parameters may describe none of them well.

This project, **QuantRegime AI**, treats the regime itself as a quantity that can be detected, modelled and used. The system (i) discovers three latent market states from price data with an unsupervised Gaussian Mixture Model, (ii) summarises how the market moves between those states with a first-order discrete-time Markov chain, (iii) trains a separate regression model for each state and compares it with a single global model, and (iv) chains these components inside a Monte Carlo engine that simulates many possible future price paths and reports risk measures such as the 95% Value-at-Risk. Everything is delivered through an interactive Streamlit dashboard in which the user can change the asset, the models, the forecast horizon and the stress scenario and immediately see the effect.

The implementation consists of approximately 1,680 lines of Python in seven source modules and a test script, published as an open repository [14]. This report documents the motivation, theory, design, implementation and evaluation of the system. In addition to describing the software, the report evaluates it critically: the regime detector is compared with known ground truth on synthetic data, and the predictive benefit of regime conditioning is examined both in-sample and on held-out data. The report describes the repository after the corrections listed in Section 5.6.

1.2 Background

1.2.1 Financial time series and their stylised facts

Asset prices P_t are non-stationary: their mean and variance change with time, so most statistical methods are applied to log-returns, $r_t = \ln(P_t/P_{t-1})$, which are approximately stationary [35]. Decades of empirical work have identified a set of properties that are shared by returns of very

different assets and markets. Cont [8] lists, among others, the absence of significant linear autocorrelation in returns, heavy tails, and volatility clustering, meaning that large moves tend to be followed by large moves of either sign. Mandelbrot [25] had already shown that the distribution of price changes is better described by heavy-tailed laws than by the normal distribution.

These properties are visible in the simulated series used throughout this report (Chapter 5). The lag-1 autocorrelation of its daily log-returns is only -0.021, whereas the lag-1 autocorrelation of the squared returns is 0.155 and remains positive at lag 5 (0.097), which is the signature of volatility clustering. The return distribution has a skewness of -0.29 and an excess kurtosis of 1.54. Volatility clustering is exactly the kind of behaviour that a regime-based description is intended to capture.

1.2.2 Market regimes

A regime is a period during which the joint behaviour of returns and volatility is approximately stable. In this project three regimes are used and given intuitive names. The **Bull** regime is characterised by positive average returns, a positive trend and comparatively low volatility. The **Neutral** regime is a consolidation phase with returns close to zero and moderate volatility. The **Bear** regime combines negative returns, a negative trend and elevated volatility. These labels are assigned after clustering by ordering the clusters on their mean return; they are descriptive statistical categories, not the formal business-cycle dates published by official agencies.

1.2.3 Markov chains

A discrete-time Markov chain is a stochastic process on a finite set of states in which the probability of the next state depends only on the current state [28]. The whole process is summarised by a transition matrix **P** whose entry p_{ij} is the probability of moving from state *i* to state *j* in one step. From **P** one can derive the long-run (stationary) distribution of time spent in each state and the expected number of steps for which the chain remains in a state before leaving it [33]. These quantities have a direct interpretation for regimes: how often the market is bullish in the long run, and how long a bearish phase typically lasts.

1.2.4 Machine learning regression

Supervised regression learns a mapping from a feature vector to a target. Here the features are technical indicators computed from past prices and the target is the next-day log-return. Three regressors of increasing flexibility are used: ridge regression, which adds an L2 penalty to least squares [20]; ridge regression on degree-2 polynomial features; and a random forest of decision trees [7]. All three are available in the scikit-learn library [29]. Because financial returns have a

very low signal-to-noise ratio, the risk of over-fitting is high, and this is a recurring theme of the evaluation in Chapter 5.

1.2.5 Monte Carlo simulation and Value-at-Risk

Monte Carlo simulation approximates the distribution of an uncertain outcome by generating many random scenarios [27]. For a price forecast this means simulating many future paths and reading off the distribution of their end points. The *Value-at-Risk* at confidence level 95% is the loss that is exceeded with probability 5% over the horizon, and is a standard summary of downside risk [23]. In this project it is estimated as the 5th percentile of the simulated horizon returns.

1.3 Motivation

The motivation for the project comes from four observations.

1. **A single global model hides structure.** A regression with one set of coefficients assumes that the relationship between indicators and returns is the same in calm and in turbulent markets. If the relationship differs between regimes, a global fit is a compromise that may be poor in both.
2. **Interpretability matters.** Deep sequence models such as LSTM networks [19] can fit complex patterns, but they are hard to inspect and need a large amount of data. A Markov chain over three named states produces a 3×3 matrix that can be read, questioned and defended directly.
3. **Forecasts must come with uncertainty.** A point prediction of tomorrow's return is of little use on its own. A fan of simulated paths and a Value-at-Risk figure communicate how wide the range of plausible outcomes is.
4. **Stress testing should be interactive.** Being able to force a sudden crash or rally into the simulation and watch the forecast fan respond helps users understand how sensitive a conclusion is to the regime assumption.

1.4 Overview of the Proposed System

Figure 1.1 shows the processing chain of QuantRegime AI. Daily price data, either downloaded from Yahoo Finance or generated by a built-in regime-switching simulator, are converted into five technical features. A Gaussian Mixture Model groups the days into three regimes. The regime sequence is then used twice: to estimate the Markov transition matrix, and to partition the data for the regime-conditioned regression models. The Monte Carlo engine combines both and feeds the dashboard.

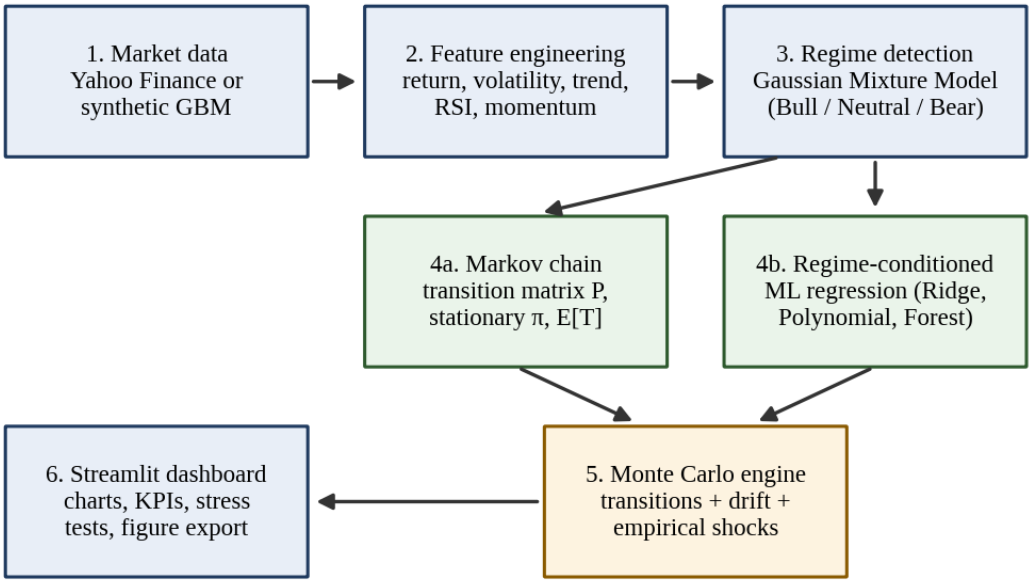


Figure 1.1: Processing chain of the QuantRegime AI system

The main capabilities of the implemented system are as follows.

- Unsupervised regime detection with a three-component Gaussian Mixture Model on standardised return, volatility and trend features, with a quantile-rule alternative.
- Estimation of the empirical transition matrix, the stationary distribution and the expected holding time of every regime.
- Regime-conditioned Ridge, Polynomial and Random Forest regressors benchmarked against a global model using R^2, RMSE, MAE and directional accuracy.
- A Monte Carlo engine that simulates 50 to 300 forward paths over 10 to 60 trading days, with a volatility multiplier and forced-regime stress tests.
- Five interactive analysis tabs, a 300-DPI four-panel figure export, and an offline synthetic-data fallback so that the demonstration never depends on a network connection.

1.5 Scope and Limitations

The project is an applied data-science and financial-engineering study. It is intended for learning, analysis and demonstration; it is not an investment product and nothing in the report constitutes trading advice. The default evaluation data are synthetic, because the synthetic generator has a known ground truth that allows the regime detector to be validated, which is impossible with real prices. The software can fetch live data when the optional *yfinance* package is installed, but all quantitative results reported here were produced on the synthetic series unless stated otherwise. Further limitations, including the use of a fixed number of regimes and a first-order Markov assumption, are discussed in Sections 3.4 and 6.3.

1.6 Organisation of the Report

Chapter 2 surveys the literature on stylised facts, regime-switching models, Markov chain theory, clustering, machine learning for return prediction, out-of-sample evidence and Monte Carlo risk measurement. Chapter 3 states the problem, lists the objectives and gives the mathematical formulation. Chapter 4 describes the implementation methodology: architecture, data, features and each module in turn, together with the verification design. Chapter 5 presents and discusses the results. Chapter 6 concludes and outlines future scope. The references and appendices follow.

CHAPTER 2: LITERATURE SURVEY

This chapter reviews the literature that underpins each component of QuantRegime AI. The survey is organised by topic rather than chronologically, and ends with a summary table and a statement of the gap that the project addresses.

2.1 Stylised Facts of Financial Returns

Mandelbrot [25] examined the distribution of speculative price changes and concluded that it has much heavier tails than the normal distribution, which challenged the Gaussian assumptions of earlier models. Fama [12] organised the discussion of market efficiency into weak, semi-strong and strong forms; in its weak form, past prices cannot be used to earn systematic excess returns, which sets a demanding benchmark for any technical-indicator forecaster. Cont [8] later summarised a set of statistical properties that appear consistently across instruments and markets, including the lack of linear autocorrelation in returns, heavy tails, volatility clustering and the asymmetry between gains and losses.

Volatility clustering motivated the family of conditional-heteroscedasticity models. Engle [11] introduced the ARCH model, in which the conditional variance depends on past squared shocks, and Bollerslev [6] generalised it to GARCH by adding lagged variances. These models describe how volatility evolves smoothly, but they do not by themselves label the market as being in a distinct qualitative state, which is the role of regime models.

2.2 Regime-Switching Models

Hamilton [16] proposed modelling a non-stationary economic series as being driven by a latent discrete state that follows a Markov chain, with a different mean growth rate in each state. His application to U.S. output growth showed that expansions and contractions could be identified statistically, and the approach became the standard Markov-switching framework. The estimation relies on filtering algorithms that are closely related to those developed for hidden Markov models in speech recognition, for which Rabiner [32] gave the widely used tutorial.

In finance, Ang and Bekaert [1] studied international asset allocation when returns follow a regime-switching process, and reported that regimes with higher volatility and higher correlation between markets matter for portfolio choice. Guidolin and Timmermann [15] fitted a multivariate regime-switching model to stock and bond returns and found that several distinct states were needed to describe the data, with consequences for optimal allocation. Kritzman, Page and Turkington [24] showed how hidden-Markov regime models can be applied to financial and economic indicators to identify regimes and to inform dynamic strategies. These studies establish that regime structure is both statistically detectable and economically relevant.

A distinction relevant to this project is between *model-based* regime estimation, in which the transition dynamics and the state-dependent distributions are estimated jointly, and a *two-stage* approach, in which states are first assigned by a clustering rule and the transition matrix is then estimated from the assigned sequence. QuantRegime AI follows the two-stage route, which is simpler to implement and to explain, at the cost of ignoring the uncertainty in the state assignment.

2.3 Markov Chain Theory

The theory used in this project is classical. Norris [28] gives a rigorous treatment of discrete-time chains, including the maximum-likelihood estimator of the transition matrix from an observed state sequence (relative transition frequencies), irreducibility and the existence and uniqueness of the stationary distribution. Ross [33] presents the same results at an applied level, including the fact that the time spent in a state before leaving it is geometrically distributed, so that its expected value is $1/(1 - p_{ii})$. The rate at which the chain forgets its starting state is governed by the second-largest eigenvalue modulus of **P**, a quantity that is reported in Chapter 5.

2.4 Unsupervised Clustering for Regime Identification

A Gaussian Mixture Model represents the data as a weighted sum of multivariate normal components. Its parameters are estimated by the expectation–maximisation algorithm of Dempster, Laird and Rubin [9], which alternates between computing the posterior probability that each observation belongs to each component and re-estimating the component parameters. Bishop [4] gives the derivation and discusses initialisation and model selection. Unlike hard-assignment K-means, the mixture model allows clusters of different shape and size through full covariance matrices, which is useful when the volatility feature has a very different spread from the return feature. The scikit-learn implementation [29] is used here.

2.5 Machine Learning for Return Prediction

Hoerl and Kennard [20] introduced ridge regression to stabilise least-squares estimates when predictors are correlated, by shrinking the coefficients towards zero. Hastie, Tibshirani and Friedman [18] discuss shrinkage, basis expansions such as polynomial features and the bias–variance trade-off in detail. Breiman [7] introduced random forests, which average many decorrelated decision trees and are robust to modest amounts of noise but can still over-fit when the signal is weak.

Deep sequence models have also been applied. Hochreiter and Schmidhuber [19] introduced the long short-term memory network, and Fischer and Krauss [13] applied LSTM networks to daily directional prediction for large-capitalisation U.S. stocks, reporting better results than memory-

free methods before transaction costs, with the advantage weakening in later years. Such models require substantial data and offer limited transparency, which is one reason this project favours simple, regularised regressors combined with an explicit regime structure.

2.6 Out-of-Sample Evidence and Over-fitting

Evidence that predictive relations survive out of sample is much weaker than evidence of in-sample fit. Welch and Goyal [37] examined a long list of variables proposed to predict the equity premium and found that most of them failed to beat a simple historical-mean benchmark when evaluated out of sample. Bailey and co-authors [3] show how the repeated testing of many model variants on the same history makes it easy to report spurious performance, and argue for a strict separation of fitting and evaluation. Pesaran and Timmermann [30] proposed a non-parametric test of whether the proportion of correctly predicted signs exceeds what would be expected by chance, which is the appropriate statistical frame for the *directional accuracy* metric used in this project.

These results shaped the evaluation design of the present work. The dashboard reports in-sample metrics for convenience; the report therefore adds a chronological hold-out evaluation and a repeated-seed robustness study so that the conclusions do not rest on a single favourable fit.

2.7 Monte Carlo Simulation and Risk Measurement

The Monte Carlo method [27] estimates expectations and quantiles by random sampling. In finance the standard model for a price is geometric Brownian motion, in which log-returns are normally distributed with constant drift and volatility [5, 21]; the synthetic data generator of this project uses a regime-switching variant of it. Efron's bootstrap [10] resamples observed values with replacement to approximate a sampling distribution without a parametric assumption; the shock generator of the simulation engine uses this idea, drawing from the empirical residuals of the active regime model. Jorion [23] defines Value-at-Risk and discusses its estimation by simulation, including the sampling error of tail quantiles, which becomes important when the number of paths is small.

2.8 Software Ecosystem

The implementation relies on the open Python scientific stack: NumPy [17] for arrays, SciPy [36] for linear algebra and statistics, pandas [26] for time-indexed data, scikit-learn [29] for clustering and regression, Matplotlib [22] for publication figures, Plotly [31] for interactive charts and Streamlit [34] for the web interface. Market data may be downloaded through the community-maintained yfinance package [2]. The relative strength index used as a feature was introduced by Wilder [38].

2.9 Summary of the Survey and Research Gap

Table 2.1: Summary of the surveyed literature and its use in this project

Work	Contribution	Use in QuantRegime AI
Mandelbrot (1963); Cont (2001)	Heavy tails and stylised facts of returns	Motivates empirical residual shocks and regime-dependent volatility
Engle (1982); Bollerslev (1986)	ARCH and GARCH volatility models	Background for volatility clustering feature
Hamilton (1989); Rabiner (1989)	Markov-switching and hidden Markov models	Conceptual basis of the latent-regime view
Ang and Bekaert (2002); Guidolin and Timmermann (2007); Kritzman et al. (2012)	Regimes in asset allocation and dynamic strategies	Economic motivation for regime conditioning
Dempster et al. (1977); Bishop (2006)	EM algorithm and Gaussian mixtures	Regime detector (GMM, full covariance)
Norris (1997); Ross (2014)	Markov chain estimation, stationarity, holding times	Transition matrix, π and E[T] module
Hoerl and Kennard (1970); Breiman (2001)	Ridge regression; random forests	Regime-conditioned regressors
Welch and Goyal (2008); Bailey et al. (2014); Pesaran and Timmermann (1992)	Out-of-sample failure and backtest over-fitting; sign test	Hold-out and multi-seed evaluation design
Metropolis and Ulam (1949); Efron (1979); Jorion (2006)	Monte Carlo, bootstrap, Value-at-Risk	Simulation engine and risk metrics

The survey shows that regime structure, Markov chain theory, clustering and regression are each mature topics, and that out-of-sample scepticism is well justified. What is less commonly packaged is a compact, transparent pipeline in which all of these elements are connected end to end, can be run interactively without a network connection, and is evaluated against a known ground truth. QuantRegime AI is an attempt to fill that gap at the scale of a semester project, and the critical evaluation in Chapter 5 is intended to show honestly how far the approach goes and where it does not.

CHAPTER 3: PROBLEM STATEMENT AND OBJECTIVES

This chapter forms the first part of the present work. It states the problem, lists the objectives with measurable outcomes, and gives the mathematical formulation of every component. The implementation methodology that realises this formulation is described in Chapter 4.

3.1 Problem Statement

Given a daily price series for a financial asset, the task is to (a) discover a small number of latent market regimes from the series without labelled data, (b) model how the market moves between regimes, (c) predict the next-day return using models that are specific to the active regime, and (d) turn these components into a probabilistic forecast of future prices that includes a quantitative measure of downside risk. The system must be interactive, must work without an internet connection, and must make every intermediate quantity inspectable.

A conventional approach fits one regressor to the whole history. Figure 3.1 illustrates why this may be inadequate. The price series, its daily log-returns and its 20-day realised volatility show alternating calm and turbulent periods: the annualised volatility ranges from about 10% to more than 45%. A model with constant coefficients treats a day from a quiet phase and a day from a turbulent phase as exchangeable observations.

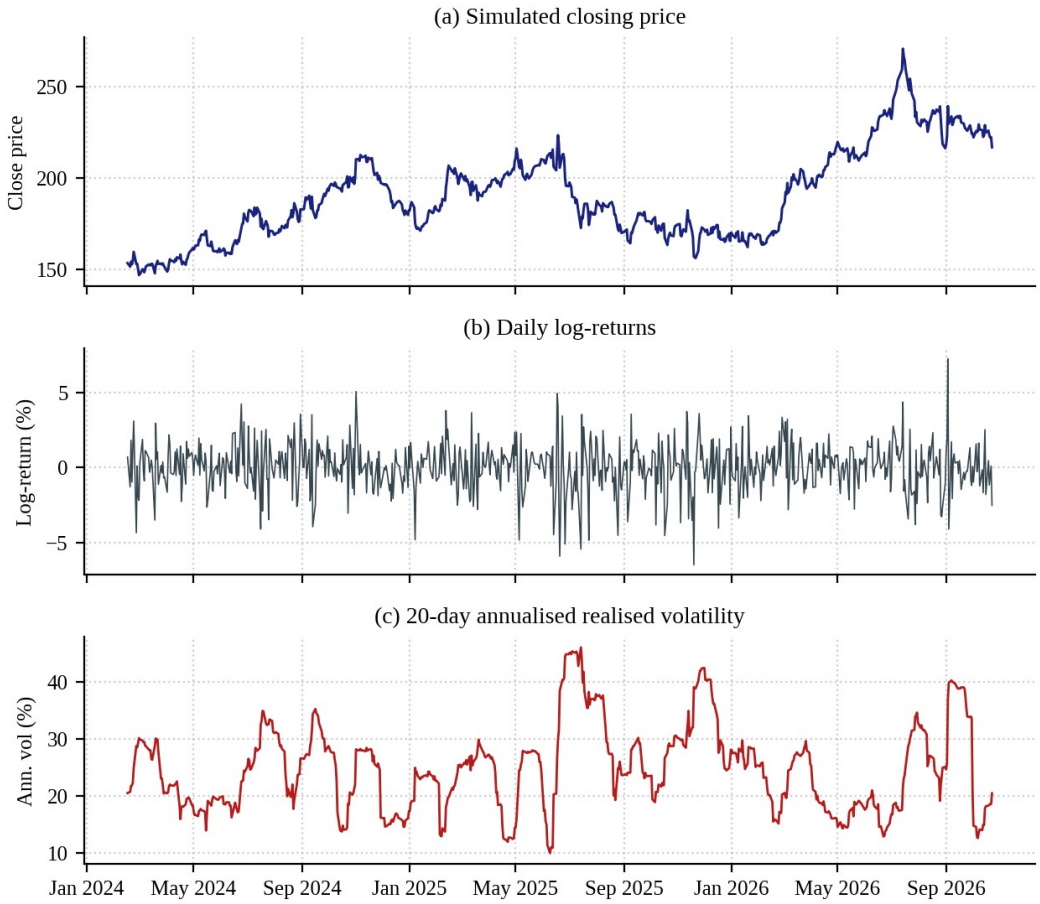


Figure 3.1: Simulated price, daily log-returns and 20-day realised volatility, showing alternating calm and turbulent periods

The investigation is organised around four research questions.

- **RQ1 (detection).** Can an unsupervised Gaussian Mixture Model on simple technical features produce interpretable regimes, and do they recover the states that generated the data when the ground truth is known?
- **RQ2 (dynamics).** Does a first-order Markov chain fitted to the detected regimes give a stable, interpretable summary through its transition matrix, stationary distribution and holding times?
- **RQ3 (prediction).** Does conditioning the regression on the regime improve next-day return prediction compared with a global model, both in-sample and on held-out data?
- **RQ4 (risk).** How do the Monte Carlo risk estimates respond to stress scenarios, and how stable are they with respect to the number of simulated paths and to the choice of regressor?

3.2 Objectives

The objectives of the project and the measurable output that demonstrates each of them are listed in Table 3.1.

Table 3.1: Project objectives and their measurable outcomes

No.	Objective	Module	Measurable outcome
1	Obtain price data from a live source or a reproducible synthetic generator	data_loader	OHLCV table, 750 rows
2	Engineer regime features from prices	regime_detector	Five indicators
3	Detect three regimes without labels	regime_detector	Regime label per day
4	Estimate Markov dynamics	markov_model	P, π, E[T]
5	Train regime-conditioned and global regressors	ml_regressor	R^2, RMSE, MAE, DA
6	Simulate forward price paths and risk	monte_carlo	Fan, VaR, P(gain)
7	Present results interactively	app, visualizations	Five-tab dashboard
8	Validate against ground truth and held-out data	test and experiments	Agreement, hold-out DA

3.3 Mathematical Formulation

3.3.1 Returns and features

Let P_t denote the closing price on trading day t. The daily log-return is

$$r_t = \ln \frac{P_t}{P_{t-1}} \tag{3.1}$$

Four further features are derived. The 20-day annualised realised volatility is $\sigma_t = \sqrt{252} \times s_t$, where s_t is the sample standard deviation of the last 20 log-returns. The trend spread compares the 20-day and 50-day simple moving averages,

$$\Delta_t = \frac{\mathrm{SMA}_t^{(20)}}{\mathrm{SMA}_t^{(50)}} - 1 \tag{3.2}$$

The relative strength index over 14 days, in the simple-average form used in the code, is

$$\mathrm{RSI}_t = 100 - \frac{100}{1 + \mathrm{RS}_t} \tag{3.3}$$

where RS_t is the mean gain divided by the mean loss over the window [38]. Finally, the 10-day momentum is $M_t = P_t/P_{t-10} - 1$. The vector of the five features is denoted $\mathbf{x}_t$.

3.3.2 Gaussian Mixture Model

Regime detection uses the standardised triple $(r_t, \sigma_t, \Delta_t)$. With $K = 3$ components the density of an observation is

$$p(x_t) = \sum_{k=1}^{K} \omega_k N(x_t; \mu_k, \Sigma_k) \tag{3.4}$$

where ω_k are the mixture weights, μ_k the means and Σ_k full covariance matrices, estimated by the expectation–maximisation algorithm [9]. The responsibility of component k for day t is

$$\gamma_{tk} = \frac{\omega_k N(x_t; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \omega_j N(x_t; \mu_j, \Sigma_j)} \tag{3.5}$$

and the day is assigned to the component with the largest responsibility. The components are then relabelled so that state 0 has the highest mean return (Bull), state 1 the middle (Neutral) and state 2 the lowest (Bear).

3.3.3 Discrete-time Markov chain

The regime sequence $S_t \in \{0, 1, 2\}$ is assumed to satisfy the first-order Markov property,

$$P(S_{t+1} = j | S_t = i, S_{t-1}, \ldots, S_0) = P(S_{t+1} = j | S_t = i) = p_{ij} \tag{3.6}$$

The maximum-likelihood estimate of the transition probabilities from the observed sequence is the relative frequency of transitions,

$$p_{ij} = \frac{N_{ij}}{\sum_{k=0}^{2} N_{ik}} \tag{3.7}$$

where N_{ij} counts the transitions from state i to state j. The stationary distribution $\boldsymbol{\pi}$ is the left eigenvector of $\mathbf{P}$ for eigenvalue 1,

$$\pi P = \pi, \quad \sum_{i=0}^{2} \pi_i = 1 \tag{3.8}$$

and the expected holding time of state i, the mean of a geometric random variable, is

$$E[T_i] = \sum_{k=1}^{\infty} k p_{ii}^{k-1}(1 - p_{ii}) = \frac{1}{1 - p_{ii}} \tag{3.9}$$

3.3.4 Regime-conditioned regression

The target is the next-day log-return $y_t = r_{t+1}$. A global ridge model solves

$$\beta^* = \arg\min_{\beta} \sum_{t=1}^{T} (y_t - x_t^T \beta)^2 + \alpha \|\beta\|^2 \tag{3.10}$$

whereas the regime-conditioned model fits a separate parameter vector for every state s using only the days on which $S_t = s$,

$$\beta_s^* = \arg\min_\beta \sum_{t:S_t=s} (y_t - x_t^T\beta)^2 + \alpha\|\beta\|^2 \tag{3.11}$$

and predicts with $f_s(\mathbf{x}_t)$. The same structure applies when ridge regression is replaced by the polynomial or random-forest regressor. The residuals of each regime model are stored and later resampled by the simulator.

3.3.5 Monte Carlo simulation and Value-at-Risk

For each of N paths and each forecast day, the next state is drawn from the row of $\mathbf{P}$ belonging to the current state (or forced to a chosen state during a stress test), the regime model gives the expected return, and an empirical residual scaled by the stress multiplier λ is added:

$$r_{t+1} = \hat{r}_{t+1} + \lambda\varepsilon_{t+1} \tag{3.12}$$

The simulated return is clipped to the interval $[-0.15, 0.15]$ and the price evolves multiplicatively,

$$P_{t+1} = P_t \exp(r_{t+1}) \tag{3.13}$$

With $R_\tau = (P_\tau - P_0)/P_0$ denoting the simulated horizon return, the 95% Value-at-Risk is the 5th percentile of its simulated distribution, $VaR_{95} = Q_{0.05}(R_\tau)$, and the probability of a gain is the share of paths with $R_\tau > 0$.

3.3.6 Evaluation metrics

For n observations with targets y_t and predictions $\hat{y}_t$ the following metrics are computed.

$$R^2 = 1 - \frac{\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{n}(y_t - \bar{y})^2} \tag{3.14}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2} \tag{3.15}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t| \tag{3.16}$$

$$\mathrm{DA} = \frac{100}{n}\sum_{t=1}^{n} I(\mathrm{sign}(y_t) = \mathrm{sign}(\hat{y}_t)) \tag{3.17}$$

The directional accuracy *DA* is the percentage of days on which the predicted and realised returns have the same sign. Under the null hypothesis of no skill its expected value is close to 50% (or to the share of positive days if a constant "up" forecast is used), and its significance can be assessed with a binomial or Pesaran–Timmermann test [30].

3.4 Assumptions and Constraints

The formulation rests on the following assumptions, whose consequences are examined in Chapter 5.

1. The number of regimes is fixed at three and is not selected from the data.
2. The regime sequence is first-order Markov and time-homogeneous, so the transition matrix does not change over the sample.
3. Within a regime the relation between features and next-day return is stable enough to be learned from the days belonging to that regime.
4. Residuals of a regime model are exchangeable, so that resampling them reproduces the shock distribution of that regime.
5. In the simulation, only the return feature is updated from step to step; the volatility, trend, RSI and momentum features are held at their last observed values.
6. The regime of each day is assigned by a model that is fitted on the complete history supplied to the dashboard. The dashboard's hold-out metrics therefore use labels that have seen the test period, and are a descriptive check and not a walk-forward test.

Non-functional requirements are that the dashboard responds within a few seconds on an ordinary laptop (achieved by caching the fitted pipeline), that it runs offline, and that the full pipeline is covered by an automated script.

CHAPTER 4: IMPLEMENTATION METHODOLOGY

This chapter describes how the formulation of Chapter 3 was implemented. It follows the order of the data flow: architecture, data, features, regime detection, Markov chain, regression, simulation and dashboard, and ends with the design of the verification experiments.

4.1 System Architecture

The application is organised in five layers (see Figure 4.1). The data layer supplies prices; the modelling layer contains the three statistical components; the simulation layer combines them; the visualisation layer turns results into charts; and the presentation layer, a Streamlit script, wires the controls to the computation. Each layer depends only on those below it, which keeps the modules independently testable.

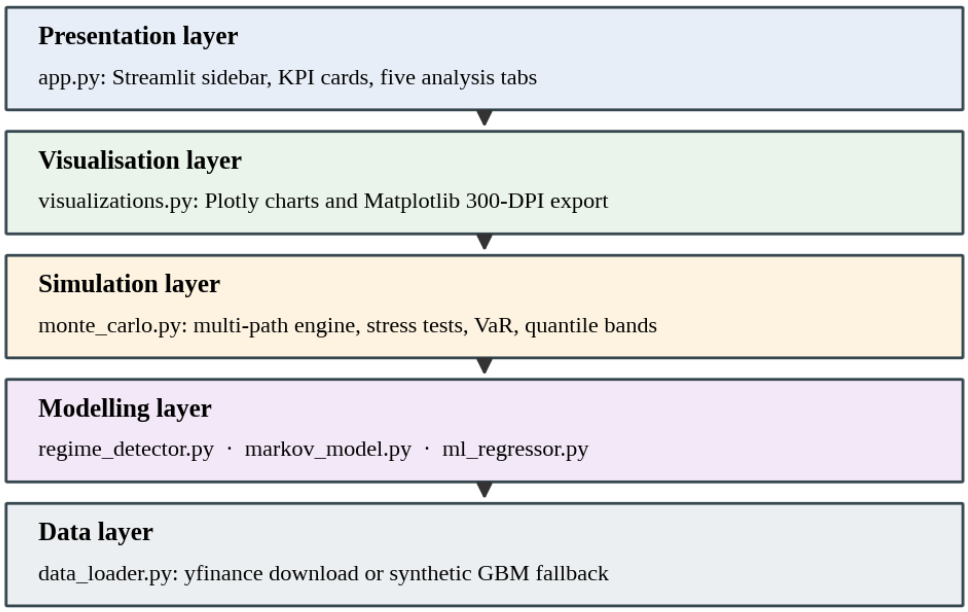


Figure 4.1: Layered architecture of QuantRegime AI

Table 4.1 lists the files of the repository with their size and responsibility. The Python sources total about 1,680 lines.

Table 4.1: Repository modules, size and responsibility

File	Lines	Responsibility
app.py	467	Streamlit dashboard: controls, cached pipeline, KPI cards, five tabs
src/data_loader.py	103	Yahoo Finance download and synthetic regime-switching generator
src/ regime_detector.py	136	Feature engineering, GMM / quantile regime labelling, regime statistics

File	Lines	Responsibility
src/ markov_model.py	115	Transition matrix, stationary distribution, holding times, path simulation
src/ml_regressor.py	158	Global and regime-conditioned regressors, residual store, evaluation
src/monte_carlo.py	113	Multi-path simulator, stress tests, quantile bands, VaR
src/ visualizations.py	540	Five Plotly charts and the Matplotlib four-panel export
test_pipeline.py	46	Headless five-stage verification script
run.sh, requirements.txt	30	One-command launcher and dependency list

4.2 Technology Stack

Table 4.2: Technology stack

Component	Minimum version	Purpose
Python	3.10	Implementation language
NumPy	1.26	Array computation, random sampling
pandas	2.0	Time-indexed data frames, rolling statistics
SciPy	1.11	Eigen-decomposition for the stationary distribution
scikit-learn	1.3	GaussianMixture, Ridge, PolynomialFeatures, RandomForestRegressor, metrics
Plotly	5.18	Interactive charts in the dashboard
Matplotlib	3.8	Publication-grade 300-DPI figure export
Streamlit	1.30	Web interface and caching
yfinance (optional)	0.2.30	Live Yahoo Finance download

All experiments reported in this document were run with Python 3.12, pandas 3.0.2 and scikit-learn 1.8.0, using the repository modules without modification.

4.3 Data Acquisition

4.3.1 Live market data

The function *fetch_market_data* tries to import *yfinance* and download the selected ticker for the chosen look-back period. The download is accepted only if it contains at least 60 rows. A multi-level column index returned by the library is flattened, rows with missing values are dropped, and the five columns Open, High, Low, Close and Volume are retained; the adjusted close is set equal

to the close. The function returns the data frame together with a flag stating whether the data are real.

4.3.2 Synthetic regime-switching generator

If the download fails, or if *yfinance* is not installed (it is commented out in *requirements.txt*), the function *generate_synthetic_market_data* supplies a reproducible substitute. A three-state Markov chain with the matrix of Table 4.3 produces a hidden regime path of 750 business days. In each state the daily log-return is drawn from a normal distribution with the state's drift and volatility, and the price is updated multiplicatively from a start price of 150, which is geometric Brownian motion with regime switching [21]. The random seed is fixed at 42, so the series is identical on every run.

Table 4.3: Parameters of the synthetic data generator

State	Drift μ (daily)	Volatility σ (daily)	To Bull	To Neutral	To Bear
0 Bull	+0.0010	0.009	0.88	0.09	0.03
1 Neutral	+0.0002	0.013	0.10	0.80	0.10
2 Bear	−0.0012	0.024	0.05	0.15	0.80

Open prices are set to the previous close, the intraday range is a random fraction of the state volatility applied to compute the high and low, and volume is log-normal and increased by 50% in the Bear state. The generator therefore has a known ground truth that real data cannot provide: the generating regime of every day is recoverable by re-running the same seed, and it is used in Section 5.2 to validate the detector.

One implementation issue was found when the generator was run for this report. In the version first reviewed it built the date index with *pd.date_range(end=today, periods=750, freq="B")*. Under pandas 3.0.2 this call returns only 749 dates when *today* falls on a weekend, so the data frame construction raised a *ValueError*; the fallback, and hence the test script, failed on Saturdays and Sundays. The corrected version rolls the end date back to the last business day with *pd.offsets.BDay().rollback*, and the generator then returns 750 rows on every day of the week (Section 5.6). The experiments of this report pinned the end date to 23 October 2026, which changes only the date labels and not any numerical result.

4.4 Feature Engineering

RegimeDetector.compute_indicators enriches the price table with the features of Table 4.4. Rolling windows produce missing values at the start of the sample (the 50-day moving average needs 50 prices), so the first 49 rows are dropped. From the 750 generated days, 701 remain for

modelling, beginning on 16 February 2024; since the last day has no next-day target, 700 rows enter the regression stage.

Table 4.4: Engineered features

Feature	Definition	Used by
Log_Return	ln(P_t / P_t−1)	GMM, regressors
Rolling_Vol_20	20-day std of returns × √252	GMM, regressors
Trend_Spread	SMA20 / SMA50 − 1	GMM, regressors
RSI	100 − 100/(1 + mean gain / mean loss), 14 days	Regressors
Mom_10	P_t / P_t−10 − 1	Regressors

4.5 Regime Detection Module

The three features Log_Return, Rolling_Vol_20 and Trend_Spread are standardised with a *StandardScaler* and clustered with scikit-learn's *GaussianMixture* using three components, full covariance matrices, five random initialisations and a fixed random state of 42. The fitted model converged on the study data. The raw cluster identifiers are then sorted by the mean log-return of their members, so that state 0 is the Bull cluster, state 1 the Neutral cluster and state 2 the Bear cluster, and the mapping is stored. An alternative *quantile* method labels a day as Bull if its return is above the 45th percentile, otherwise Bear if its 20-day volatility is above the 65th percentile, otherwise Neutral; in the corrected version these labels are used as they are and are not re-sorted by mean return, which could scramble the volatility-based Bear rule.

get_regime_statistics summarises each regime by its number of days, share, annualised return computed as (exp(mean daily return × 252) − 1), annualised volatility, and a Sharpe ratio computed with an assumed risk-free rate of 3%. These are the figures shown in the "Regime-Partitioned Price Action" tab.

4.6 Markov Chain Module

MarkovChainModel.fit loops over the regime sequence, increments the count matrix for every consecutive pair, and normalises each row to obtain the maximum-likelihood transition matrix of Equation (3.7). A row without observations is made absorbing to avoid division by zero. The stationary distribution is obtained from *scipy.linalg.eig* applied to the transpose of **P**: the eigenvector whose eigenvalue is closest to 1 is taken and normalised to sum to one *before* negative numerical noise is clipped. The order matters, because the sign of an eigenvector is arbitrary: in the version first reviewed the vector was clipped first, so an all-negative eigenvector became all zeros and the division produced NaN (Section 5.6). Holding times follow Equation

(3.9), with a cap of 999 days if the diagonal probability is practically 1. The class also provides *simulate_path* for sampling state sequences and two helper functions that return the matrix and a theoretical summary as data frames for the dashboard.

4.7 Regime-Conditioned Regression Module

RegimeMLRegressor builds the dataset from the five features and the next-day return target. Three model types are available (see Table 4.5); each is a scikit-learn *Pipeline* that begins with a *StandardScaler*.

Table 4.5: Regressor configurations

Type	Configuration
Ridge	StandardScaler followed by Ridge regression, $\alpha = 1.0$
Polynomial	StandardScaler, PolynomialFeatures (degree 2, no bias), Ridge with $\alpha = 1.0$
Random Forest	StandardScaler followed by RandomForestRegressor, 75 trees, maximum depth 5, random state 42

During *fit*, one global pipeline is trained on all rows and one pipeline per regime on the rows of that regime. If a regime has ten rows or fewer, the global model is used for it instead. For each regime the in-sample residuals are stored for the simulator. *predict_next* returns the prediction of the model of the requested regime, and *evaluate* computes R^2, RMSE, MAE and directional accuracy of the global model and of the regime-conditioned predictions, and returns them as a comparison table. It is important to note that *evaluate* scores the data it is given. In the version first reviewed the dashboard called it on the same rows that were used for fitting, so its metrics were in-sample; the corrected dashboard fits a separate model on the first 70% of the rows and scores it on the last 30% (a chronological hold-out), while the model used for the forecast is still fitted on all rows. The regime labels remain those of a GMM fitted on the full history, so this is a descriptive check and not a walk-forward test. The stricter protocol of Section 5.4 refits the detector on the training window as well.

4.8 Monte Carlo Simulation Module

MonteCarloSimulator.run_simulation implements the procedure of Algorithm 4.1 and the flow chart of Figure 4.2.

Algorithm 4.1: Regime-chained Monte Carlo simulation

Step	Operation
1	Inputs: latest price P_0, current regime S_0, latest feature vector x, number of paths N, horizon τ, multiplier λ, optional forced regime s* and shock duration k.
2	For each path n = 1 … N, set price ← P_0, state ← S_0.
3	For each day d = 1 … τ: if s *is set and d ≤ k, then next* ← *s*; otherwise draw next from the row of P belonging to state.
4	Compute the expected return r̂ with the regression model of regime next applied to x.
5	Draw a residual ε uniformly from the stored residuals of regime next (a Normal draw with σ of 0.008, 0.010 or 0.015 is used if fewer than six residuals exist) and set r = r̂ + λ ε.
6	Clip r to [−0.15, 0.15]; set price ← price × exp(r); record price and regime; set x[0] ← r and state ← next.
7	After all paths: compute the median and the 10th, 25th, 75th and 90th percentiles per day, the terminal returns, the 5th percentile of the terminal returns (VaR) and the share of positive terminal returns.

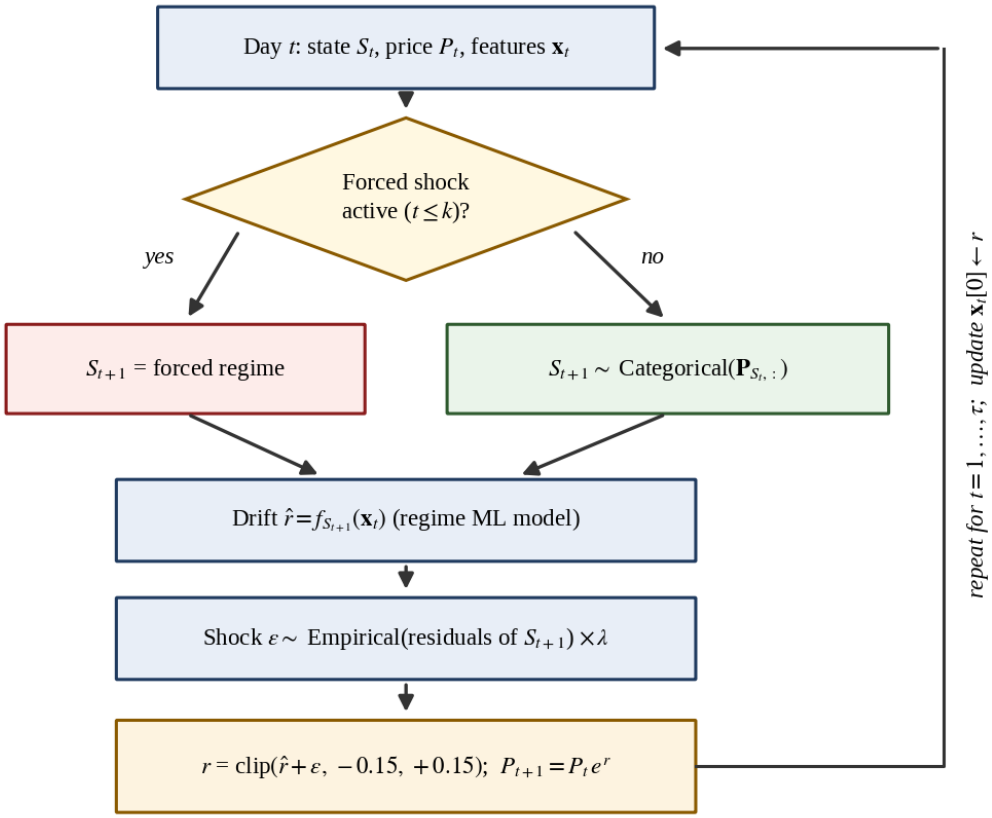


Figure 4.2: Flow chart of one simulated day

Two features of the implementation deserve emphasis. First, the drift of day $t + 1$ is computed with the regression model of the *new* state, whereas the models were trained to predict the return

of the following day from the features of a day in the *current* state; the two coincide when the regime persists. Second, only the first feature (the return) is updated along a path; the other four are held fixed. Both are modelling simplifications, and their consequences are discussed in Section 5.5. The simulator does not set a random seed, so repeated runs give slightly different outputs; the experiments in this report set the global NumPy seed explicitly before each run.

4.9 Dashboard and Visualisation Layer

The Streamlit application *app.py* is a single script that is re-executed on every interaction. The expensive part, comprising data loading, detection, Markov estimation and regression training, is wrapped in a function decorated with *st.cache_data* (time-to-live 3,600 seconds), so that moving a simulation slider only re-runs the Monte Carlo step. The sidebar controls are summarised in Table 4.6 and the five tabs in Table 4.7.

Table 4.6: Sidebar controls of the dashboard

Control	Options	Default
Market asset	SPY, NVDA, AAPL, TSLA, BTC-USD, custom ticker, synthetic demo	SPY
Historical look-back	1y, 2y, 3y, 5y	2y
Regime classifier	Gaussian Mixture Model, quantile rule	GMM
ML regressor	Ridge, polynomial (degree 2), random forest	Ridge
Forecast horizon	10 to 60 trading days, step 5	30
Simulated paths	50 to 300, step 25	150
Volatility multiplier	0.5 to 2.5, step 0.1	1.0
Shock scenario	None, Black Swan (Bear forced), Surge (Bull forced)	None

Table 4.7: Analysis tabs of the dashboard

Tab	Content
Stochastic Forecast Fan	Fan chart with median, 50% and 80% bands; terminal-return histogram; probability of gain, P10, P90
Markov Chain Engine	State-transition network, transition matrix, stationary distribution, holding times, formula explainer
Regime-Partitioned Price Action	Candlestick chart with regime bands; per-regime performance table
ML Architecture and Hold-out Check	Global versus regime-conditioned metrics on the last 30% of rows, with a caption stating the limits of the check
Faculty Viva Defence Guide	Prepared questions and answers, including a limitations answer; 300-DPI Matplotlib figure export

A header row of key indicators shows the latest close, the median target price, the hold-out directional accuracy of the regime model in points relative to the global model, and the 95% Value-at-Risk, labelled as the 5th-percentile simulated return over the horizon (a left-tail return, not an expected shortfall). A badge states whether live data or the synthetic fallback is in use. The visualisation module draws the charts with Plotly; the export function *generate_matplotlib_report_figure* produces a four-panel figure of the price with regimes, the transition matrix, the forecast fan and the stationary distribution at 300 dpi.

4.10 Verification and Experimental Design

Verification has two parts. The first is the repository's own *test_pipeline.py*, which runs the five stages (data loading, regime detection, Markov estimation, regression, simulation) and asserts basic invariants such as the shape of the transition matrix. The second is a set of additional experiments written for this report that import the repository modules and leave them unchanged. They are summarised in Table 4.8.

Table 4.8: Experiments run for the evaluation

Experiment	Procedure	Reported in
E1 Detection	Fit GMM on the full series; compare with the generator's hidden regimes (same seed)	Section 5.2
E2 Markov	Fit chain on detected states; compare P, π, E[T] with the generator	Section 5.3
E3 In-sample ML	Fit and score on all 700 rows for three regressors	Section 5.4
E4 Hold-out ML	Chronological 70/30 split with detector and regressors fitted on the training part only	Section 5.4
E5 Robustness	Repeat E1 and E3 to E4 on 30 independent synthetic series	Section 5.4
E6 Monte Carlo	Base run, stress scenarios, convergence with N, regressor sensitivity	Section 5.5
E7 Software checks	Test script, pipeline loop over all classifier, regressor and data-size combinations, headless runs of the dashboard, stationary-distribution test on random chains	Section 5.6

Algorithm 4.2: Chronological hold-out protocol

Step	Operation
1	Compute the features for the whole series (all indicators use only past data).
2	Split the 701 feature rows chronologically: the first 70% are the training set, the last 30% the test set.
3	Fit the scaler and the GMM on the training rows only; assign test rows to regimes with the fitted model and its stored cluster mapping.
4	Fit the global and regime-conditioned regressors on the training rows; the test rows are never used for fitting.
5	Score both predictors on the test rows with R^2, RMSE, MAE and directional accuracy.

For E5 the generator's hard-coded seed was overridden from outside, without editing the file, to obtain independent series with seeds 1 to 30; the ground-truth regimes of each series were regenerated from the same seed. Monte Carlo runs use the base setting of the dashboard (150 paths, 30 days, ridge regressor) unless stated.

4.11 Deployment

The application is started with *run.sh*, which creates a virtual environment on first use, installs *requirements.txt* and launches *streamlit run app.py*. The dashboard is then served at *http://localhost:8501*. The installation and reproduction steps are listed in Appendix B.

CHAPTER 5: RESULTS AND DISCUSSION

This chapter reports the outcome of the experiments of Section 4.10 and discusses them against the four research questions. Unless stated otherwise the data are the synthetic series with seed 42. All numbers are produced by the repository modules; the code that generated them is described in Appendix B.

5.1 Dataset Summary

Table 5.1 summarises the series. It spans 750 business days, of which 701 remain after the indicator warm-up. The close rises from about 149 to 217 over the period, with a maximum drawdown visible in Figure 3.1. The daily log-return has a mean of about 0.05% and a standard deviation of 1.62%, equivalent to an annualised volatility of about 25.7%.

Table 5.1: Summary statistics of the synthetic series (seed 42)

Quantity	Value
Observations generated	750
Period	11 Dec 2023 to 23 Oct 2026
First and last close	148.96 and 216.72
Rows after indicator warm-up	701 (from 16 Feb 2024)
Rows used for regression	700
Mean daily log-return	0.050%
Standard deviation of daily log-return	1.62%
Skewness / excess kurtosis	−0.29 / 1.54
Lag-1 autocorrelation of returns	−0.021
Lag-1 autocorrelation of squared returns	0.155
Share of days with a positive next-day return	54.6%

5.2 Regime Detection Results

5.2.1 Detected regimes

Figure 5.1 shades the price with the regimes found by the Gaussian Mixture Model. The GMM converged, and its mixture weights are 0.388, 0.440, 0.173. The Bull regime covers 276 days (39.4%), the Neutral regime 321 days (45.8%) and the Bear regime 104 days (14.8%).

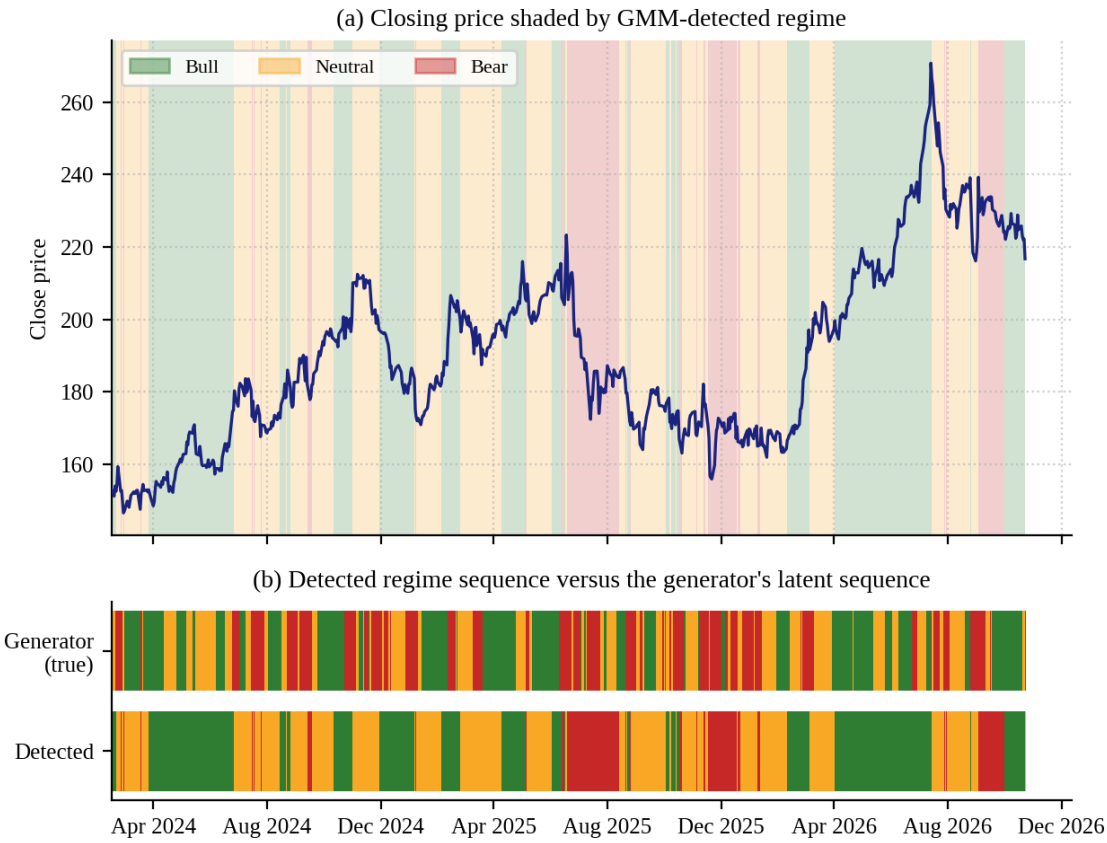


Figure 5.1: Price shaded by detected regime (a) and detected versus generator regime sequence (b)

Table 5.2 reproduces the regime statistics table of the dashboard and Table 5.3 gives the average feature values of each regime. The ordering is economically coherent: from Bull to Bear the mean return and trend spread fall, the 20-day volatility rises from 17.5% to 37.0%, and the RSI falls from 58.0 to 45.6. Figure 5.2 shows that the clusters are separated mainly along the volatility axis, with the return and trend axes separating the Bull cluster from the rest.

Table 5.2: Regime statistics produced by the dashboard

Regime	Days	Share	Ann. return	Ann. volatility	Sharpe
Bull	276	39.4%	+139.93%	19.87%	6.89
Neutral	321	45.8%	+25.87%	23.16%	0.99
Bear	104	14.8%	-88.65%	38.54%	-2.38

Table 5.3: Mean feature values by detected regime

Regime	Return (%/day)	Volatility (%)	Trend spread (%)	RSI	Mom_10 (%)
Bull	0.347	17.5	1.52	58.0	1.85
Neutral	0.091	26.9	1.13	52.4	0.57
Bear	−0.863	37.0	−1.80	45.6	−1.94

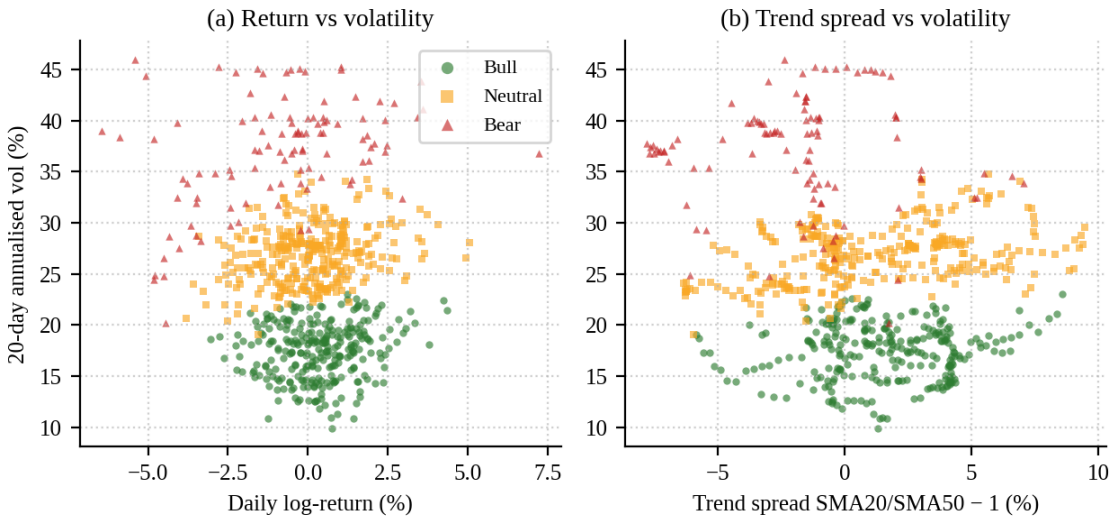


Figure 5.2: Detected regimes in feature space: return versus volatility (a) and trend spread versus volatility (b)

A caution is needed in reading Table 5.2. The annualised returns (+139.93% for Bull and -88.65% for Bear) and the Sharpe ratio of 6.89 are descriptive. Because the same-day return is one of the clustering inputs, the clusters are partly defined by return, so the Bull cluster by construction contains the better days. These numbers describe the regimes; they are not the performance of an implementable strategy.

5.2.2 Agreement with the generating regimes

Because the generator is seeded, its hidden regime of every day can be regenerated, which allows the detector to be validated directly (RQ1). Table 5.4 cross-tabulates the generator regimes with the detected ones, and Figure 5.3 displays the same counts. The overall agreement is 40.1%, against roughly one third expected from a random assignment. Using measures that do not depend on the labelling, the adjusted Rand index is 0.026 and the normalised mutual information is 0.035, both close to zero. Re-matching the labels optimally with the Hungarian algorithm raises the agreement only to 40.4%.

Table 5.4: Generator regimes (rows) versus detected regimes (columns), number of days

Generator regime	Bull	Neutral	Bear	Total
Bull	123	122	23	268
Neutral	102	99	22	223
Bear	51	100	59	210

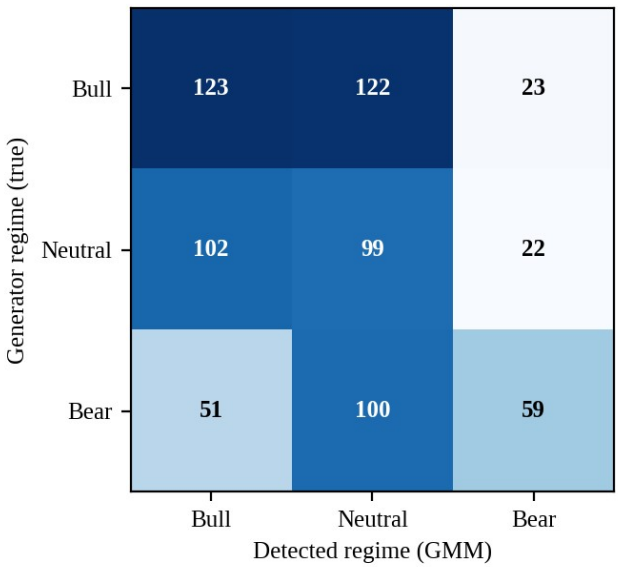


Figure 5.3: Confusion matrix of detected versus generator regimes

The reason is visible in Table 5.5. The generator's regimes are short: the mean spell lasts 4.2 to 8.4 days. The detector's main features, however, are the 20-day volatility and the 20/50-day trend spread, which average over several such spells and therefore vary slowly. The detected regimes are consequently much more persistent (mean spells of 14.5, 9.4 and 5.0 days) and trace the slow-moving volatility and trend state of the price path rather than the day-to-day hidden state. They are meaningful descriptions of the observed path, but on this data they are not a recovery of the generating process.

Table 5.5: Spell lengths of the generator and detected regimes (days)

Regime	Gen. mean	Gen. spells	Gen. longest	Det. mean	Det. spells	Det. longest
Bull	8.4	32	26	14.5	19	74
Neutral	4.2	53	16	9.4	34	32
Bear	5.5	38	13	5.0	21	40

This answers RQ1 in two parts. The detector produces interpretable, persistent and economically ordered regimes, but it does not recover the generator's hidden states when those change faster than the feature windows can follow. On real data, where no ground truth exists, the regimes should be presented as a descriptive classification of trend and volatility conditions. The multi-seed study of Section 5.4 confirms the result: across 30 independent series the mean agreement is 35.5% (range 15.8% to 51.8%).

5.3 Markov Chain Results

Table 5.6 gives the transition counts obtained from the 701 detected states (700 transitions), and Figure 5.4 the resulting matrix beside the generator's own matrix. Figure 5.5 is the state-transition

network in the form of the dashboard's Markov tab, with node size proportional to the stationary probability.

Table 5.6: Observed transition counts between detected regimes

From \ To	Bull	Neutral	Bear	Total
Bull	257	15	4	276
Neutral	16	287	17	320
Bear	2	19	83	104

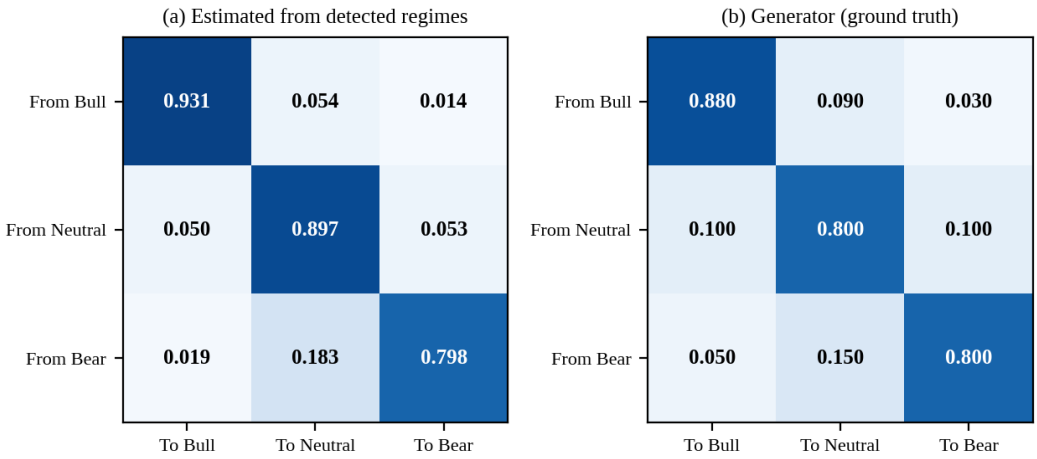


Figure 5.4: Estimated transition matrix (a) and generator transition matrix (b)

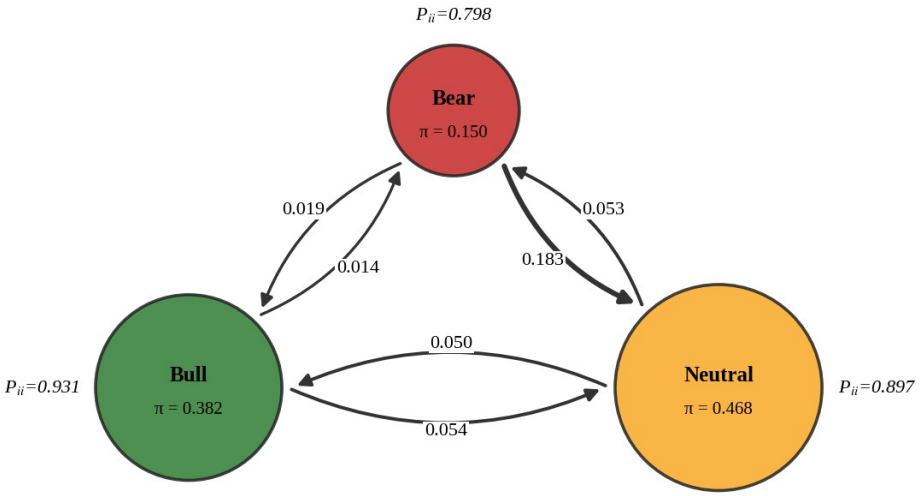


Figure 5.5: Markov state-transition network of the detected regimes

Every regime is strongly persistent: the diagonal entries are 0.931, 0.897, 0.798 for Bull, Neutral and Bear. Departures from the Bull and Neutral regimes are rare and mostly go to the adjacent state; only 0.014 of Bull days are followed by a Bear day. The Bear regime is the least persistent and exits mainly to Neutral (0.183), a recovery pattern. Table 5.7 lists the derived quantities and Figure 5.6 compares them with the generator. The expected holding times are 14.5, 9.7, 5.0 days.

Table 5.7: Stationary distribution and expected holding time

Regime	π (est.)	π (gen.)	Occupancy	E[T] (est.)	E[T] (gen.)
Bull	0.382	0.400	0.394	14.5	8.3
Neutral	0.468	0.360	0.458	9.7	5.0
Bear	0.150	0.240	0.148	5.0	5.0

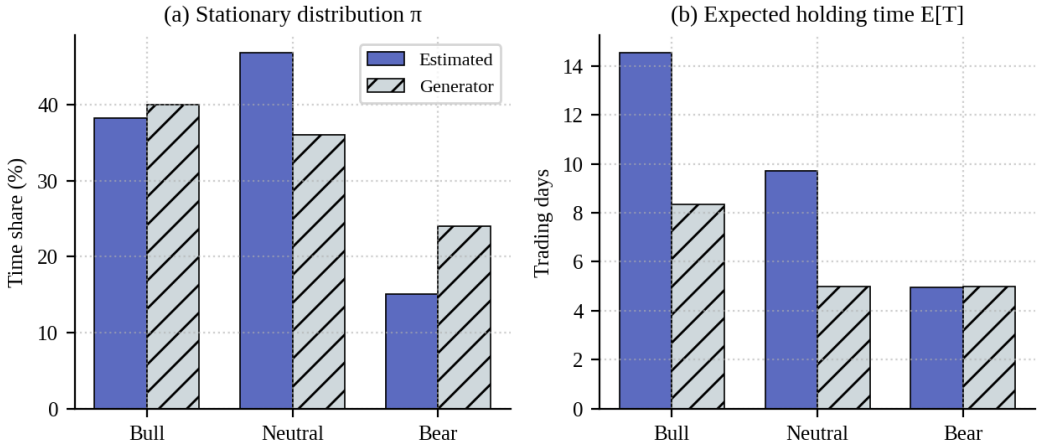


Figure 5.6: Stationary distribution (a) and expected holding time (b), estimated versus generator

The stationary distribution is internally consistent with the data, since it is close to the empirical share of days in each regime (column Occupancy). The moduli of the eigenvalues of the estimated matrix are 1.000, 0.889, 0.737; the second-largest governs convergence, so the chain forgets its starting state with a time constant of about 8.5 days. Compared with the generator, the estimated matrix differs by a Frobenius norm of 0.142 (largest single-entry error 0.097). The estimated holding times are longer than the generator's, and the estimated stationary share of the Bear regime (0.150) is lower than the generator's (0.240). Both effects follow from the smoothness of the detected regimes discussed above, because a chain fitted to smoother labels is more persistent.

One benchmark needs to be kept in mind when the chain is used for one-step prediction. The most probable next state is in every row the current state, so predicting the next regime with the argmax of the transition row gives the "no change" forecast; it is correct on 89.6% of days. This high figure reflects persistence and not predictive skill. This answers RQ2: the chain provides a stable and readable summary of the detected regimes, but its parameters inherit the properties of the detector.

5.4 Regime-Conditioned Regression Results

5.4.1 In-sample comparison

Table 5.8 and Figure 5.7 show the comparison as the dashboard originally displayed it, in which every model is fitted and scored on the same 700 observations. For the ridge model the regime-conditioned predictor has a higher R^2 (0.0362 against 0.0122) and a lower RMSE, but its directional accuracy (55.1%) is slightly below that of the global model (56.0%). For the polynomial and random-forest models, regime conditioning raises both R^2 and directional accuracy. The random forest reaches an in-sample R^2 of 0.435 and a directional accuracy of 72.6%.

Table 5.8: In-sample comparison of global and regime-conditioned predictors

Model	Predictor	R^2	RMSE	MAE	DA (%)
Ridge	Global	0.0122	0.01607	0.01202	56.0
	Regime-conditioned	0.0362	0.01588	0.01187	55.1
Polynomial	Global	0.0378	0.01586	0.01178	57.9
	Regime-conditioned	0.1180	0.01519	0.01153	60.4
Random Forest	Global	0.2429	0.01407	0.01077	69.3
	Regime-conditioned	0.4354	0.01215	0.00946	72.6

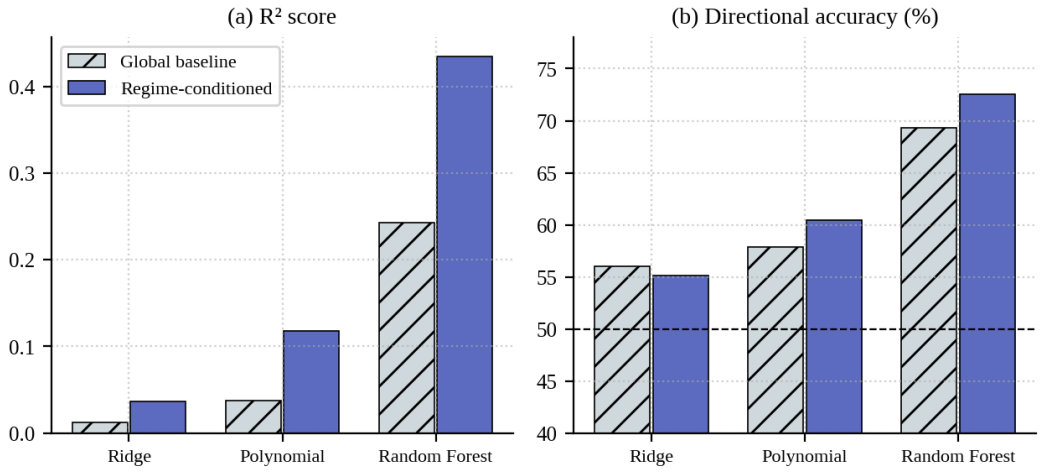


Figure 5.7: In-sample R^2 (a) and directional accuracy (b) of the global and regime-conditioned models

These in-sample figures should be read with care. The share of positive next-day returns in the sample is 54.6%, so a constant "up" forecast would score about 55% on directional accuracy, and the ridge result of 55% to 56% is essentially at that level. The random-forest scores reflect memorisation: 75 trees of depth 5 can fit 700 noisy points far better than any stable relation would justify. A binomial test of the random-forest directional accuracy against 50% gives a vanishingly small p-value, but the test is not valid because the model was scored on its own training data.

5.4.2 Chronological hold-out

Following Algorithm 4.2, the first 490 feature rows were used for fitting and the last 210 rows (from 2 January 2026) for testing. The detector was fitted on the training window only and applied to the test window. Table 5.9 gives the results. The share of positive days in the test window is 55.7%.

Table 5.9: Hold-out comparison (training rows 1 to 490, test rows 491 to 700)

Model	Predictor	R^2	RMSE	MAE	DA (%)
Ridge	Global	0.0021	0.01507	0.01147	52.9
	Regime-conditioned	−0.0184	0.01523	0.01135	59.0
Polynomial	Global	−0.0515	0.01547	0.01160	51.4
	Regime-conditioned	−0.5565	0.01883	0.01283	55.2
Random Forest	Global	−0.0885	0.01574	0.01193	53.3
	Regime-conditioned	−0.1017	0.01584	0.01187	53.3

On held-out data the picture changes. The R^2 of every predictor is close to zero or negative; the random-forest R^2 is −0.089 (global) and −0.102 (regime-conditioned), and the regime-conditioned polynomial model, with an R^2 of −0.557, is much worse than a constant forecast. The ridge regime-conditioned model reaches a directional accuracy of 59.0% against 52.9% for the global model, a difference of 6.2 percentage points; but its R^2 is lower. With 210 test days the standard error of a directional accuracy near 50% is about 3.4 percentage points, so a single split cannot distinguish a real improvement from luck.

The corrected dashboard uses a simpler protocol than Algorithm 4.2: the regime labels come from the GMM fitted on the full history, the regressors are fitted on the first 490 of the 701 rows and scored on the remaining 211 (from 6 January 2026), and the forecast model is refitted on all rows. Table 5.10 shows what it displays. For the ridge model the regime-conditioned predictor shows a directional accuracy of 58.6% against 52.9% for the global model, a gap of 5.7 points, which is the figure on the indicator card. This gap should be read as a single-split diagnostic. It is smaller than the standard deviation of 4.1 points of the same quantity across 30 independent series (Section 5.4.3), the R^2 values are close to zero, and for the polynomial and random-forest models the regime-conditioned predictor is not better on R^2.

Table 5.10: Hold-out metrics as displayed by the corrected dashboard (GMM labels from the full history)

Model	R^2 global	R^2 regime	DA global (%)	DA regime (%)	Gap (points)
Ridge	0.002	0.004	52.9	58.6	+5.7
Polynomial	−0.052	−0.369	51.4	53.8	+2.4
Random Forest	−0.089	−0.139	53.3	52.9	−0.5

5.4.3 Robustness over 30 synthetic series

To avoid drawing a conclusion from one fortunate or unfortunate sample, experiments E1 and E3 to E4 were repeated on 30 independent series (seeds 1 to 30). Table 5.11 and Figure 5.8 summarise the outcome; the individual seeds are in Appendix C.

Table 5.11: Ridge model over 30 independent synthetic series

Quantity	Mean	Std	Min	Max
DA global, in-sample (%)	52.7	1.5	50.3	56.9
DA regime, in-sample (%)	54.5	1.8	50.9	57.6
DA global, hold-out (%)	49.7	3.7	41.4	60.0
DA regime, hold-out (%)	49.3	3.8	43.8	56.2
Share of positive days, hold-out (%)	50.9	3.1	44.3	57.1
R^2 global, in-sample	0.008	0.006	0.001	0.025
R^2 regime, in-sample	0.032	0.013	0.013	0.060
R^2 global, hold-out	−0.019	0.024	−0.092	0.016
R^2 regime, hold-out	−0.074	0.065	−0.315	0.013

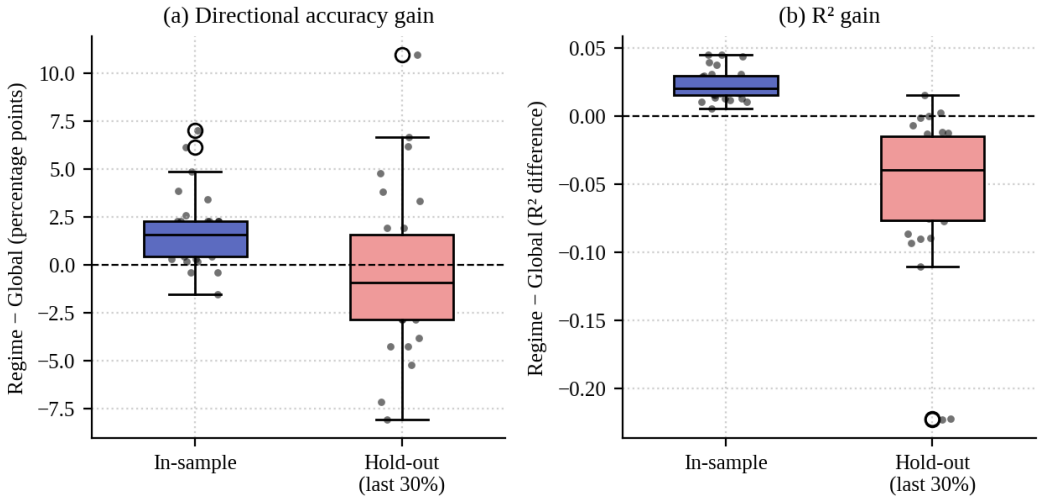


Figure 5.8: Gain of regime conditioning over the global model across 30 series, in-sample and on hold-out

In-sample, regime conditioning improves directional accuracy by 1.8 percentage points on average (it wins in 27 of 30 series) and raises the R^2 from 0.008 to 0.032. On the hold-out window the average difference is −0.37 percentage points with a standard deviation of 4.1; the regime model is better in only 13 of 30 series, and a paired t-test gives $p = 0.63$. The hold-out R^2 is negative for both predictors and is lower for the regime-conditioned model (−0.074 against −0.019). The regime-conditioned directional accuracy exceeds that of a constant "up" forecast in only 10 of 30 series.

This answers RQ3. The apparent benefit of regime conditioning that the dashboard shows is an in-sample effect: it is reproduced in 27 of 30 series on the training data and disappears on held-out data, where the additional parameters of the regime-conditioned model (one model per regime, each fitted on fewer rows) are a likely source of extra estimation error. The finding agrees with the literature, in which in-sample predictability of returns commonly fails to survive out-of-sample evaluation [37] and in which model comparisons on the fitting sample are prone to over-fitting [3]. The synthetic generator contains no relation between technical features and future returns other than the small regime drift (0.1% per day or less against volatilities of 0.9% to 2.4%), so the null result is also what the data design would lead one to expect. Claims about improved prediction should therefore be limited to the in-sample comparison, and any statement about forecasting skill requires a walk-forward evaluation on real data.

5.5 Monte Carlo Simulation Results

5.5.1 Base forecast

The base run uses the default dashboard setting: 150 paths, 30 trading days and the ridge regime-conditioned model, starting from the last close of 216.72 with the market in the Neutral regime. Figure 5.9 shows the forecast fan and Figure 5.10 the distribution of the simulated horizon returns; the key outputs are in Table 5.12.

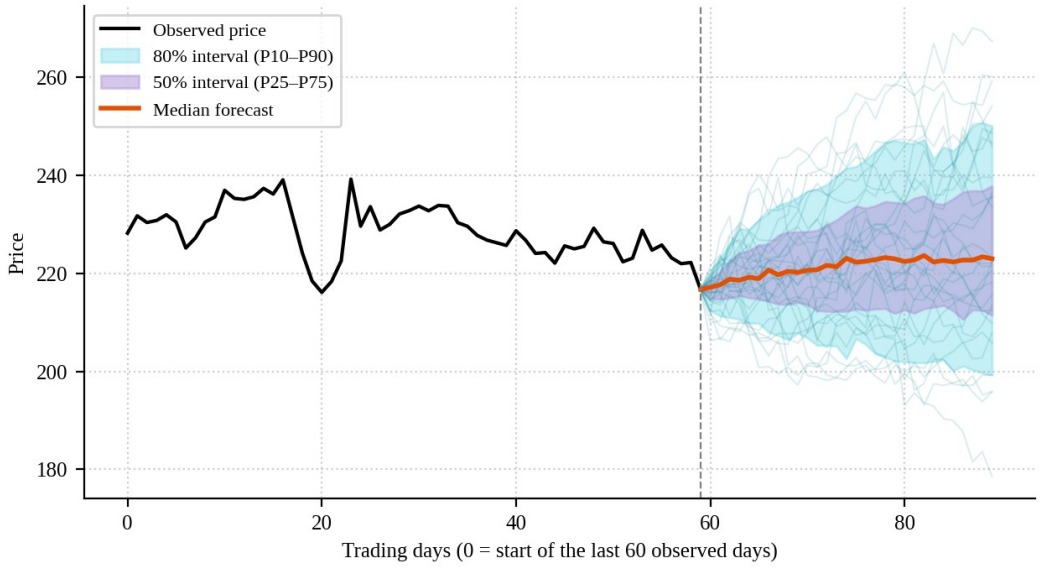


Figure 5.9: Monte Carlo forecast fan: 150 paths, 30 trading days

Table 5.12: Base-run Monte Carlo output

Quantity	Value
Median terminal price	223.00 (+2.90%)
10th and 90th percentile of terminal price	199.25 and 250.11
Mean and standard deviation of horizon return	+4.02% and 9.76%
95% Value-at-Risk (5th percentile of horizon return)	−9.78%
Probability of a positive horizon return	66.7%

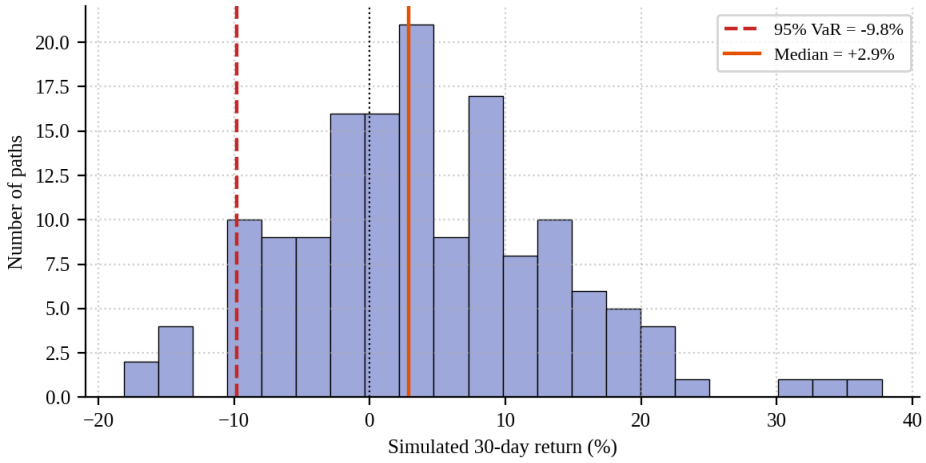


Figure 5.10: Distribution of simulated 30-day returns with median and 95% VaR

The fan widens roughly with the square root of time, as expected from independent shocks. The simulated spread of horizon returns (standard deviation 9.8%) is consistent with the historical daily volatility of 1.62% scaled over 30 days, which is about 8.9%. The positive median is produced by the regression drift, which, given a −2.5% return on the last day and a low RSI of 36.4, is a positive "mean reversion" drift in the Bull and Neutral models (+0.39% and +0.28% per

day at the last feature vector) and negative in the Bear model (−0.49%). Since Section 5.4 found no out-of-sample skill, this drift should not be interpreted as a forecast of gain; the more reliable content of the fan is its width.

5.5.2 Stress scenarios

Table 5.13 and Figure 5.11 compare five scenarios obtained with the same random seed. Forcing the Bear regime for the first five days (the "Black Swan" control) moves the median horizon return from +2.9% to −3.5% and roughly doubles the Value-at-Risk, from −9.8% to −20.2%; the probability of a gain falls from 67% to 35%. Scaling the shocks by λ = 1.5 and 2.0 widens the tail almost proportionally: the VaR becomes −15.0% and −20.1%, that is 1.53 and 2.05 times the base value.

Table 5.13: Monte Carlo results under stress scenarios (150 paths, 30 days)

Scenario	Median return (%)	VaR 95 (%)	Std (%)	P(gain) (%)	P90 price
Natural Markov flow	+2.90	−9.78	9.8	66.7	250.1
Black Swan (forced Bear, 5 days)	−3.53	−20.19	10.7	35.3	238.4
Surge (forced Bull, 5 days)	+0.50	−9.12	7.6	52.0	242.0
Volatility x1.5	+3.92	−14.97	14.1	62.0	265.0
Volatility x2.0	+5.46	−20.07	18.7	61.3	277.4

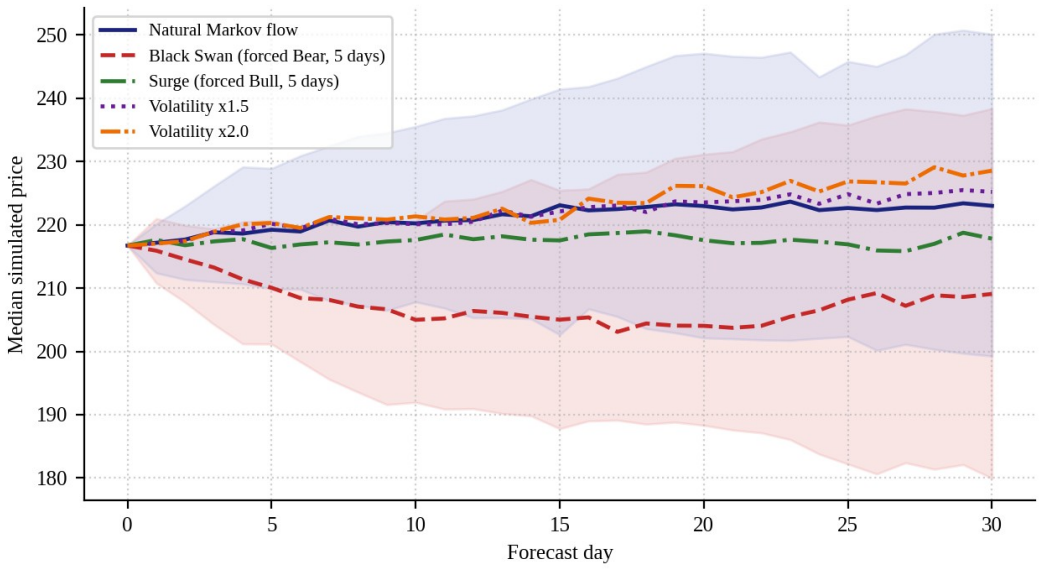


Figure 5.11: Median simulated price under five scenarios with P10–P90 bands for the natural and Black Swan runs

The "Surge" control behaves less intuitively. Forcing the Bull regime for five days gives a median return of +0.5%, lower than the +2.9% of the unforced run. The result is not sampling noise: with 800 paths and a different seed the median returns are +3.0% (natural), +0.9% (Bull forced) and

−3.1% (Bear forced), so the ordering persists. A plausible contributor is the simplification noted in Section 4.8: only the return feature is updated along a path, while volatility, trend, RSI and momentum stay at their last observed values, so the regime models are applied to feature vectors that are inconsistent with the regime they represent. The cause was not isolated in this study; it is listed as a point for future work. The downside scenarios, which are the ones most relevant to risk, behave as intended.

5.5.3 Sampling error and sensitivity to the model

A 95% VaR is a tail quantile, so with 150 paths it is determined by only about seven of them. Table 5.14 and Figure 5.12 show how it changes with the number of paths. With seed 2026 the estimate moves from −6.6% at 50 paths to −14.4% at 600 paths, and five other seeds at the default 150 paths give 12.0% to 13.4% (losses, mean 12.7%). The default setting therefore produces a noisy and, on average, optimistic tail estimate; 1,000 paths or more are advisable when the VaR figure is to be quoted. The median, by contrast, is stable to about one percentage point.

Table 5.14: Effect of the number of simulated paths (seed 2026)

Paths N	Median return (%)	VaR 95 (%)	Std (%)	P(gain) (%)
50	+3.18	−6.62	7.4	68.0
100	+3.37	−8.16	8.9	64.0
150	+2.90	−9.78	9.8	66.7
300	+2.34	−13.34	9.8	61.3
600	+2.96	−14.39	10.3	62.3

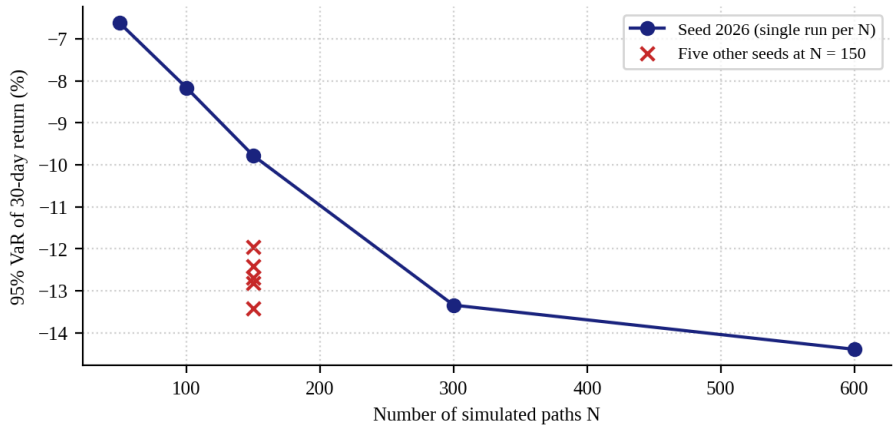


Figure 5.12: 95% VaR as a function of the number of simulated paths

The forecast also depends strongly on which regressor is chosen. Table 5.15 repeats the base run (100 paths) with the three regressors. The median horizon return is +3.4% for ridge, −1.3% for the random forest and −8.7% for the polynomial model, and the probability of a gain ranges from 24% to 64%. The sign of the central forecast is therefore a product of the model choice, an

example of model risk. Because none of the regressors showed skill on held-out data, the interpretation that is best supported is that the simulator is a tool for exploring risk under assumptions and not a price predictor. This answers RQ4.

Table 5.15: Sensitivity of the Monte Carlo output to the regressor (100 paths)

Regressor	Median return (%)	VaR 95 (%)	P(gain) (%)
Ridge	+3.37	−8.16	64.0
Polynomial	−8.67	−24.48	24.0
Random Forest	−1.31	−12.53	41.0

5.5.4 Exported academic figure

Figure 5.13 is the four-panel figure produced by the dashboard's export function for the base run. It combines the regime-shaded price, the transition matrix, the forecast fan and the stationary distribution in one publication-style graphic that can be inserted into a report or a presentation. The date labels of the first panel were reformatted for legibility; the content is otherwise as produced by the function.

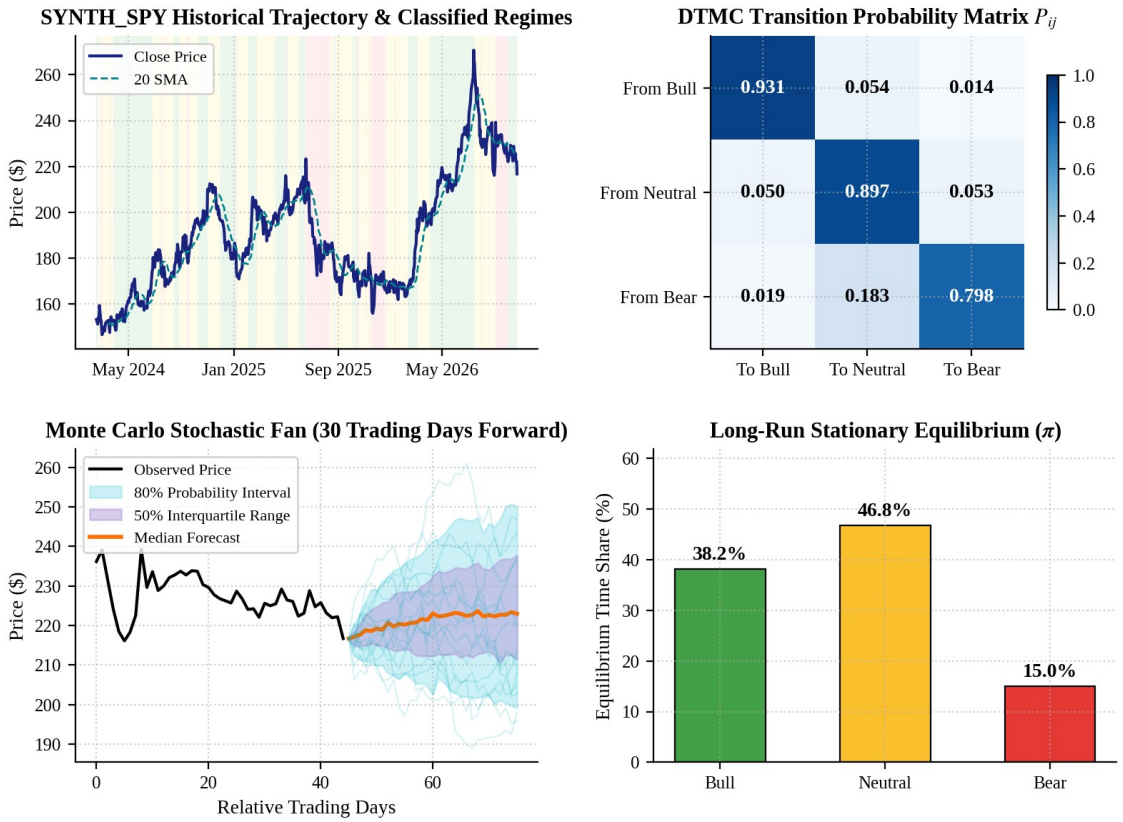


Figure 5.13: Four-panel figure exported by generate_matplotlib_report_figure for the synthetic series

5.6 Testing Results and Issues Found

The repository script *test_pipeline.py* was executed on the version first reviewed. With the weekday workaround of Section 4.3.2 it completed all five stages and printed the success message; without it, run on a Sunday, the first stage failed with a *ValueError*. After the correction the script runs unmodified on every day of the week and gives the same stage results, which are listed in Table 5.16.

Table 5.16: Results of the repository test script

Stage	Result
1. Data loading	750 rows loaded; live data flag False (synthetic fallback)
2. Regime detection	Three regimes; Bull 276, Neutral 321, Bear 104 days
3. Markov chain	π = [0.382, 0.468, 0.150]; E[T] = [14.5, 9.7, 5.0] days; matrix shape 3 × 3
4. Regression	Ridge: R^2 0.0122 (global), 0.0362 (regime); comparison table produced
5. Monte Carlo	50 paths, 20 days: median terminal price 219.51, VaR −9.79%

Two software defects were found during the study. The first is the weekend date failure of Section 4.3.2. The second is in the stationary distribution of the Markov module: the sign of an eigenvector returned by the linear-algebra routine is arbitrary, and the original code clipped negative entries to zero before normalising, so an all-negative eigenvector became a vector of zeros and the division gave NaN. On the project data the sign happens to be positive, which is why the test script passed. In a test on 300 randomly generated three-state chains the original code returned an invalid distribution for 193 of them, whereas the corrected order (normalise, then clip) returned none. The same defect is the likely cause of a run of the dashboard that stopped with the message "Axis limits cannot be NaN or Inf" before the correction. A review of the repository additionally pointed out several mismatches between claims and code. All of these were corrected in a patch; Table 5.17 lists them.

Table 5.17: Corrections applied to the repository

No.	Issue found	Correction
1	Synthetic generator raised ValueError on weekends (749 dates)	End date rolled back to the last business day
2	Stationary distribution invalid for negative eigenvector (NaN)	Normalise before clipping
3	Dashboard ML metrics were in-sample; tab titled "Validation"	70/30 chronological hold-out; tab renamed; caption added

No.	Issue found	Correction
4	VaR card captioned "Maximum Expected Loss"	Relabelled as 5th-percentile simulated return (left tail)
5	Viva answers claimed Bull = high drift and low volatility, "no overfitting", and "outperforms"	Reworded to ranking by average return and softened; limitations answer added
6	Quantile labels re-sorted by mean return; K-Means mentioned in a docstring	No re-sorting for the quantile rule; docstring corrected
7	Histogram title hard-coded 150 paths; README claimed a drastic accuracy gain	Title uses the actual path count; README states the range and the small or absent held-out benefit
8	Three different project names in README, page title and launcher	QuantRegime AI used throughout
9	Frozen features undocumented; python-docx missing from requirements	Limitation commented in the code; optional dependency noted

The corrected version was verified in four ways, summarised in Table 5.18. The pipeline loop exercises the new hold-out logic for every classifier and regressor on three data sizes; the headless runs drive the real dashboard. Only the default configuration and three further option combinations were run through the full dashboard, because each run takes several minutes; the remaining combinations were covered by the pipeline loop only.

Table 5.18: Verification of the corrected version

Check	Scope	Result
Test script	Five pipeline stages	Passes; same figures as above
Pipeline loop	2 classifiers × 3 regressors × 3 data sizes (about 210, 400 and 750 days)	18 combinations, no invalid output
Dashboard, headless	Default setting plus GMM / Ridge / no shock / 2y; Quantile / Random Forest / Black Swan / 2y; GMM / Polynomial / Surge / 1y	No exceptions
Weekend dates	Saturday, Sunday and Monday	750 rows each
Stationary distribution	300 random three-state chains	Original 193 invalid; corrected 0

5.7 Summary of Findings

Table 5.19: Answers to the research questions

RQ	Finding
RQ1	Detected regimes are interpretable, ordered and persistent, but agree with the generator's hidden states only at 40% (ARI 0.03), because the feature windows are longer than the generator's regimes.
RQ2	The Markov chain gives a stable, readable summary (high self-transition probabilities, π close to occupancy); it inherits the persistence of the detector, and its one-step "prediction" equals persistence.
RQ3	Regime conditioning improves in-sample fit and (for ridge, over 30 series) directional accuracy by 1.8 points, but gives no reliable gain on held-out data (mean −0.4 points, $p = 0.63$). The single split shown by the corrected dashboard (+5.7 points for ridge) is within this spread.
RQ4	Stress tests respond sensibly to downside shocks and volatility scaling; the VaR estimate is noisy at 150 paths, and the forecast direction depends on the regressor.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project designed, implemented and evaluated QuantRegime AI, an interactive system that combines unsupervised regime detection, a discrete-time Markov chain, regime-conditioned regression and Monte Carlo simulation. The software works end to end, runs offline through a reproducible synthetic data generator, and exposes every intermediate quantity through a dashboard with five analysis tabs, stress-test controls and a publication-quality figure export.

The evaluation, which was designed to test the approach and not merely to demonstrate it, leads to the following conclusions. First, the Gaussian Mixture Model produces regimes that are interpretable and ordered by return, volatility and trend, but on synthetic data with a known ground truth they agree with the generator's hidden states at only 40%, since the regime features average over longer windows than the generator's regimes last. Second, the Markov chain fitted to these regimes is stable and readable, with strong persistence (expected holding times of 14.5, 9.7, 5.0 days for Bull, Neutral and Bear), and its stationary distribution agrees with the observed time shares. Third, regime conditioning improves the in-sample fit of the regressors, but in a chronological hold-out and over 30 independent series it gives no reliable improvement in directional accuracy or R^2. Fourth, the Monte Carlo engine responds sensibly to downside stress and volatility scaling, but its Value-at-Risk estimate is noisy at the default 150 paths and its central forecast depends on the regressor chosen. Finally, the review and testing of the repository exposed two software defects and several claim mismatches, which were corrected and re-verified.

The system is therefore best described as an interpretable framework for regime analysis and scenario-based risk exploration. The evidence supports its use for describing market conditions and for illustrating the width and downside of plausible outcomes; it does not support claims that regime conditioning yields a better price or direction forecast. Reporting this result candidly, together with the evidence that supports it, is regarded as a contribution of the work in its own right.

6.2 Contributions

1. A modular, tested Python implementation of a regime-chained forecasting pipeline of about 1,680 lines.
2. An interactive dashboard with live Markov-state graph, forecast fan, candlestick regime bands, model benchmark and stress-test sliders.

3. A validation of the regime detector against the known hidden regimes of a seeded generator, including label-independent agreement measures.
4. A chronological hold-out and a 30-series robustness study that separate in-sample fit from generalisation.
5. An analysis of the sampling error of the simulated Value-at-Risk and of its sensitivity to the regressor.
6. Identification and correction of two software defects (a weekend-dependent failure of the synthetic generator and a sign-dependent failure of the stationary distribution) and of several mismatches between claims and code, with the corrected version re-tested (Section 5.6).

6.3 Limitations

- All quantitative results use synthetic data in which returns are independent given the regime; real markets may behave differently and the conclusions must be re-examined on real series.
- The number of regimes (three) is fixed and the regime sequence is assumed to be first-order Markov and time-homogeneous.
- The two-stage approach ignores the uncertainty of the regime assignment, and the GMM is fitted on the whole history supplied, which lets later data influence earlier labels.
- The dashboard's hold-out check is a single chronological split that uses regime labels fitted on the full history, so it is descriptive and noisy. The simulation updates only the return feature along a path and applies the model of the new state to the features of the current state.
- The 150-path default gives a noisy tail estimate, and shocks are resampled from in-sample residuals, which understate out-of-sample error.
- Transaction costs, liquidity and position sizing are not modelled, and no trading strategy is evaluated.

6.4 Future Scope

1. **Walk-forward evaluation inside the dashboard.** Replace the single 70/30 split by an expanding-window or rolling evaluation in which the detector is also refitted on each training window, report confidence intervals, and add a sign test such as that of Pesaran and Timmermann [30].
2. **Model-based regime estimation.** Fit a hidden Markov or Markov-switching model by the Baum–Welch or Hamilton filter [16, 32] so that regime probabilities, not hard labels,

drive the regression and the simulation, and select the number of regimes with an information criterion.

3. **Real-data validation.** Install *yfinance* and repeat the study on several liquid assets and crypto-currencies over different periods and, if possible, compare the detected regimes with independently dated market episodes.
4. **Consistent simulation state.** Update all five features along each simulated path and train the regime models so that the conditioning state matches the one used in the simulation.
5. **Better tail estimation.** Raise the default path count, report standard errors of the VaR, add expected shortfall, and use heavier-tailed or GARCH-type shocks [6].
6. **Automated testing.** Add the test script, the weekend-date check and the random-chain test of the stationary distribution to a continuous-integration workflow, and extend them to the full set of dashboard option combinations.
7. **Comparison with sequence models.** Benchmark the framework against an LSTM [19] under the same hold-out protocol.
8. **Strategy layer.** Convert regime probabilities into position rules and evaluate them net of transaction costs on an out-of-sample basis.
9. **Deployment.** Package the application in a container or host it on Streamlit Community Cloud, and add pinned dependency versions.

REFERENCES

1. Ang, A. and Bekaert, G. (2002) 'International Asset Allocation with Regime Shifts', *Review of Financial Studies*, Vol.15, No.4, pp.1137–1187.
2. Aroussi, R. (n.d.) *yfinance: Download Market Data from Yahoo! Finance's API*, from web site https://github.com/ranaroussi/yfinance accessed on 25/10/2026.
3. Bailey, D.H., Borwein, J.M., López de Prado, M. and Zhu, Q.J. (2014) 'Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance', *Notices of the American Mathematical Society*, Vol.61, No.5, pp.458–471.
4. Bishop, C.M. (2006) *Pattern Recognition and Machine Learning.* Springer, New York.
5. Black, F. and Scholes, M. (1973) 'The Pricing of Options and Corporate Liabilities', *Journal of Political Economy*, Vol.81, No.3, pp.637–654.
6. Bollerslev, T. (1986) 'Generalized Autoregressive Conditional Heteroskedasticity', *Journal of Econometrics*, Vol.31, No.3, pp.307–327.
7. Breiman, L. (2001) 'Random Forests', *Machine Learning*, Vol.45, No.1, pp.5–32.
8. Cont, R. (2001) 'Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues', *Quantitative Finance*, Vol.1, No.2, pp.223–236.
9. Dempster, A.P., Laird, N.M. and Rubin, D.B. (1977) 'Maximum Likelihood from Incomplete Data via the EM Algorithm', *Journal of the Royal Statistical Society: Series B*, Vol.39, No.1, pp.1–38.
10. Efron, B. (1979) 'Bootstrap Methods: Another Look at the Jackknife', *Annals of Statistics*, Vol.7, No.1, pp.1–26.
11. Engle, R.F. (1982) 'Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation', *Econometrica*, Vol.50, No.4, pp.987–1007.
12. Fama, E.F. (1970) 'Efficient Capital Markets: A Review of Theory and Empirical Work', *Journal of Finance*, Vol.25, No.2, pp.383–417.
13. Fischer, T. and Krauss, C. (2018) 'Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions', *European Journal of Operational Research*, Vol.270, No.2, pp.654–669.
14. Grey00000 (2026) *regis: QuantRegime AI – Markov Chain & Machine Learning Asset Forecaster.* Source code repository, from web site https://github.com/Grey00000/regis accessed on 25/10/2026.

15. Guidolin, M. and Timmermann, A. (2007) 'Asset Allocation under Multivariate Regime Switching', *Journal of Economic Dynamics and Control*, Vol.31, No.11, pp.3503–3544.

16. Hamilton, J.D. (1989) 'A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle', *Econometrica*, Vol.57, No.2, pp.357–384.

17. Harris, C.R. et al. (2020) 'Array Programming with NumPy', *Nature*, Vol.585, pp.357–362.

18. Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning*, 2nd edn. Springer, New York.

19. Hochreiter, S. and Schmidhuber, J. (1997) 'Long Short-Term Memory', *Neural Computation*, Vol.9, No.8, pp.1735–1780.

20. Hoerl, A.E. and Kennard, R.W. (1970) 'Ridge Regression: Biased Estimation for Nonorthogonal Problems', *Technometrics*, Vol.12, No.1, pp.55–67.

21. Hull, J.C. (2018) *Options, Futures, and Other Derivatives*, 10th edn. Pearson, New York.

22. Hunter, J.D. (2007) 'Matplotlib: A 2D Graphics Environment', *Computing in Science & Engineering*, Vol.9, No.3, pp.90–95.

23. Jorion, P. (2006) *Value at Risk: The New Benchmark for Managing Financial Risk*, 3rd edn. McGraw-Hill, New York.

24. Kritzman, M., Page, S. and Turkington, D. (2012) 'Regime Shifts: Implications for Dynamic Strategies', *Financial Analysts Journal*, Vol.68, No.3, pp.22–39.

25. Mandelbrot, B. (1963) 'The Variation of Certain Speculative Prices', *Journal of Business*, Vol.36, No.4, pp.394–419.

26. McKinney, W. (2010) 'Data Structures for Statistical Computing in Python', *Proceedings of the 9th Python in Science Conference*, pp.56–61.

27. Metropolis, N. and Ulam, S. (1949) 'The Monte Carlo Method', *Journal of the American Statistical Association*, Vol.44, No.247, pp.335–341.

28. Norris, J.R. (1997) *Markov Chains.* Cambridge University Press, Cambridge.

29. Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, Vol.12, pp.2825–2830.

30. Pesaran, M.H. and Timmermann, A. (1992) 'A Simple Nonparametric Test of Predictive Performance', *Journal of Business & Economic Statistics*, Vol.10, No.4, pp.461–465.

31. Plotly Technologies Inc. (2015) *Collaborative Data Science.* Plotly Technologies Inc., Montréal, from web site https://plot.ly accessed on 25/10/2026.

32. Rabiner, L.R. (1989) 'A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition', *Proceedings of the IEEE*, Vol.77, No.2, pp.257–286.

33. Ross, S.M. (2014) *Introduction to Probability Models*, 11th edn. Academic Press, Oxford.

34. Streamlit Inc. (2026) *Streamlit Documentation*, from web site https://docs.streamlit.io accessed on 25/10/2026.

35. Tsay, R.S. (2010) *Analysis of Financial Time Series*, 3rd edn. Wiley, Hoboken, NJ.

36. Virtanen, P. et al. (2020) 'SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python', *Nature Methods*, Vol.17, No.3, pp.261–272.

37. Welch, I. and Goyal, A. (2008) 'A Comprehensive Look at the Empirical Performance of Equity Premium Prediction', *Review of Financial Studies*, Vol.21, No.4, pp.1455–1508.

38. Wilder, J.W. (1978) *New Concepts in Technical Trading Systems.* Trend Research, Greensboro, NC.

APPENDIX A: MODULE AND INTERFACE REFERENCE

Table A.1: Public classes and functions of the repository

Item	Purpose
fetch_market_data(ticker, period, fallback_if_offline)	Returns (data frame, is_real_data); falls back to synthetic data if the download fails
generate_synthetic_market_data(ticker, n_days, start_price)	Seeded regime-switching OHLCV generator (seed 42)
RegimeDetector(n_regimes, method)	Holds the scaler, the GMM and the cluster mapping
RegimeDetector.compute_indicators(df)	Adds Log_Return, Rolling_Vol_20, Trend_Spread, RSI and Mom_10
RegimeDetector.fit_predict(df)	Fits the detector and returns the data with Regime, Regime_Name and Regime_Color
RegimeDetector.get_regime_statistics(df)	Per-regime days, share, annualised return and volatility, Sharpe ratio
MarkovChainModel.fit(regime_series)	Estimates transition counts, matrix P, stationary π and holding times
MarkovChainModel.simulate_path(state, steps)	Samples a future state sequence
RegimeMLRegressor(model_type, alpha)	Ridge, polynomial or random-forest regime models and a global model
RegimeMLRegressor.fit(df), predict_next(x, regime), evaluate(df)	Training, one-step prediction and metric comparison
MonteCarloSimulator(markov, ml, n_simulations, forecast_days)	Simulation engine
MonteCarloSimulator.run_simulation(...)	Returns paths, quantile bands, terminal returns, VaR and probability of gain
create_*_chart(...) and generate_matplotlib_report_figure(...)	Plotly charts and the 300-DPI four-panel figure

APPENDIX B: INSTALLATION AND REPRODUCTION

B.1 Running the application

1. Install Python 3.10 or later.
2. Clone the repository from https://github.com/Grey00000/regis.
3. Run *./run.sh* (Linux and macOS), or create a virtual environment, run *pip install -r requirements.txt* and then *streamlit run app.py*.
4. Open *http://localhost:8501* in a browser. Select "Synthetic Demo (Offline)" in the sidebar for the dataset used in this report.
5. For live data, additionally run *pip install yfinance*.

B.2 Reproducing the experiments

The experiments of Chapter 5 were produced by scripts that import the repository modules without modifying them. The settings were as follows: synthetic series with seed 42 (and seeds 1 to 30 for the robustness study) and 750 business days ending 23 October 2026; Gaussian Mixture Model with three components, full covariance, five initialisations and random state 42; ridge $\alpha = 1.0$; random forest of 75 trees with depth 5; chronological hold-out with 70% training; Monte Carlo runs with 150 paths, 30 days, a global NumPy seed of 2026, a five-day forced shock and the stated volatility multipliers. Because the generator's end date depends on the current date, the date was pinned to 23 October 2026 (Section 4.3.2); the numerical results do not depend on the date.

APPENDIX C: RESULTS FOR THE 30 SYNTHETIC SERIES

Table C.1: Per-seed results for the ridge model (DA in %, hold-out = last 30% of rows)

Seed	Agree. (%)	DA glob. in	DA reg. in	DA glob. out	DA reg. out	Up (%)	R^2 glob. out	R^2 reg. out
1	26.5	54.3	54.6	45.7	46.2	46.7	−0.092	−0.315
2	38.9	52.4	50.9	46.2	50.0	49.0	−0.007	−0.080
3	29.8	53.7	54.7	51.4	47.1	48.6	−0.005	−0.018
4	44.1	52.9	52.4	47.1	44.3	51.0	−0.011	−0.097
5	32.1	51.4	57.6	49.5	55.7	51.0	0.014	0.013
6	47.2	52.0	56.9	51.4	50.0	51.4	−0.017	−0.017
7	25.2	54.6	55.0	50.5	51.0	51.9	−0.007	−0.013
8	40.5	52.0	54.0	46.2	48.1	50.5	−0.017	−0.050
9	40.8	50.3	57.3	45.7	44.8	47.1	−0.057	−0.095
10	15.8	55.0	55.3	51.4	56.2	56.7	−0.026	−0.104
11	47.5	54.0	57.4	46.7	53.3	44.3	−0.010	−0.007
12	35.9	52.3	52.4	49.0	49.5	50.5	−0.027	−0.064
13	42.9	52.1	52.3	52.4	50.0	46.2	0.004	−0.218
14	38.9	54.0	56.3	60.0	55.7	52.9	0.010	−0.029
15	31.2	52.0	54.6	51.9	48.1	51.9	−0.001	−0.013
16	41.8	52.1	53.4	48.1	45.2	48.6	−0.051	−0.036
17	51.8	50.7	53.0	51.4	48.6	51.4	−0.012	−0.048
18	36.1	56.9	56.4	52.9	53.3	52.9	−0.019	−0.073
19	25.1	51.3	53.6	54.8	52.9	51.0	−0.021	−0.068
20	23.5	52.7	54.3	48.6	49.0	56.2	−0.049	−0.089
21	35.9	51.6	53.9	49.0	43.8	53.8	−0.013	−0.124
22	31.5	51.1	51.6	46.2	44.3	57.1	−0.028	−0.121
23	30.0	53.3	55.3	54.3	46.2	47.6	−0.000	−0.090
24	43.2	52.3	53.7	41.4	52.4	47.6	−0.028	−0.048
25	47.1	51.0	53.3	45.2	47.1	50.0	−0.006	−0.018
26	33.0	52.0	53.6	52.4	45.2	51.4	−0.010	−0.054
27	24.0	54.3	55.6	51.9	55.2	52.9	0.016	−0.038
28	36.4	52.9	53.7	50.0	50.0	49.5	−0.033	−0.108
29	30.7	52.0	55.9	46.2	44.3	55.2	−0.062	−0.093
30	38.1	54.4	56.7	51.9	51.0	52.9	−0.015	−0.104

APPENDIX D: SOURCE CODE

This appendix lists the complete source code of the repository, 1,707 lines in 10 files, in the corrected version described in Section 5.6 (branch quantregime-fixes of https://github.com/Grey00000/regis). The files are given in dependency order: configuration and test files first, then the modules of the data, modelling, simulation and visualisation layers, and finally the dashboard application. Every line is numbered; a long line continues on the next row without a number. The listings are set in a monospaced font at 8 pt so that the code keeps its original layout.

D.1 requirements.txt

```
numpy>=1.26.0
pandas>=2.0.0
scipy>=1.11.0
scikit-learn>=1.3.0
matplotlib>=3.8.0
streamlit>=1.30.0
plotly>=5.18.0
# Optional for live yahoo finance data:
# yfinance>=0.2.30
# Optional, only to regenerate the Word report in report/:
# python-docx>=1.1.0
```

D.2 run.sh

```
#!/usr/bin/env bash
# Quick launcher for QuantRegime AI
set -e

DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" >/dev/null 2>&1 && pwd )"
cd "$DIR"

if [ ! -d ".venv" ]; then
    echo "Creating virtual environment..."
    python3 -m venv .venv
    echo "Installing required dependencies..."
    ./.venv/bin/pip install -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host
mirrors.aliyun.com -r requirements.txt
fi

echo "============================================================"
echo " Starting QUANTREGIME AI Dashboard..."
echo " Open your browser to http://localhost:8501"
echo "============================================================"
./.venv/bin/streamlit run app.py
```

D.3 test_pipeline.py

```
import sys
from src.data_loader import generate_synthetic_market_data, fetch_market_data
from src.regime_detector import RegimeDetector, STATE_LABELS
from src.markov_model import MarkovChainModel
from src.ml_regressor import RegimeMLRegressor
from src.monte_carlo import MonteCarloSimulator

def test_full_pipeline():
    print("[1/5] Testing data loading...")
    df, is_real = fetch_market_data("AAPL", period="1y", fallback_if_offline=True)
    print(f"       Loaded {len(df)} rows. Is live data: {is_real}")
    assert len(df) > 50, "Dataframe too small"

    print("[2/5] Testing regime detection (GMM)...")
    detector = RegimeDetector(n_regimes=3, method="gmm")
```

```
    classified_df = detector.fit_predict(df)
    stats = detector.get_regime_statistics(classified_df)
    print(f"       Detected regimes successfully:\n{stats}")
    assert "Regime" in classified_df.columns

    print("[3/5] Testing Markov Chain transition matrix & stationary distribution...")
    markov = MarkovChainModel(n_states=3, state_names=STATE_LABELS)
    markov.fit(classified_df["Regime"])
    print(f"       Stationary distribution π: {markov.stationary_distribution}")
    print(f"       Expected holding times: {markov.expected_holding_times}")
    assert markov.transition_matrix.shape == (3, 3)

    print("[4/5] Testing ML Regression models...")
    ml = RegimeMLRegressor(model_type="ridge")
    ml.fit(classified_df)
    eval_res = ml.evaluate(classified_df)
    print(f"       Model Evaluation Comparison:\n{eval_res['comparison_df']}")
    assert "comparison_df" in eval_res

    print("[5/5] Testing Monte Carlo Simulation...")
    latest_price = float(classified_df["Close"].iloc[-1])
    latest_state = int(classified_df["Regime"].iloc[-1])
    latest_feats = classified_df[ml.feature_cols].iloc[-1].values
    mc = MonteCarloSimulator(markov, ml, n_simulations=50, forecast_days=20)
    sim_res = mc.run_simulation(latest_price, latest_state, latest_feats)
    print(f"       Simulated 50 paths. Terminal median: ${sim_res['median'][-1]:.2f}")
    print(f"       95% VaR: {sim_res['var_95']:.2f}%")
    print("ALL TESTS PASSED SUCCESSFULLY! Ready for presentation.")

if __name__ == "__main__":
    test_full_pipeline()
```

D.4 src/data_loader.py

```
import numpy as np
import pandas as pd
from typing import Tuple, Optional
import datetime

def generate_synthetic_market_data(
    ticker: str = "SYNTH_SPY",
    n_days: int = 750,
    start_price: float = 150.0
) -> pd.DataFrame:
    """
    Generates realistic multi-regime financial price series using
    Geometric Brownian Motion with regime shifts.
    Guarantees the dashboard works seamlessly offline or without API access.
    """
    np.random.seed(42)
    # Roll weekends back to the last business day: pd.date_range(end=<Sat/Sun>, freq="B")
    # can return n_days - 1 dates on recent pandas, which breaks the DataFrame build.
    end_date = pd.offsets.BDay().rollback(pd.Timestamp(datetime.date.today()))
    dates = pd.date_range(end=end_date, periods=n_days, freq="B")

    # 3 regimes: 0: Bull (high drift, low vol), 1: Neutral (low drift, med vol), 2: Bear (neg drift, high vol)
    regime_params = [
        {"mu": 0.0010, "sigma": 0.009},  # Bull
        {"mu": 0.0002, "sigma": 0.013},  # Neutral
        {"mu": -0.0012, "sigma": 0.024}  # Bear
    ]
    # Synthetic regime transition matrix
    trans_matrix = np.array([
        [0.88, 0.09, 0.03],
        [0.10, 0.80, 0.10],
        [0.05, 0.15, 0.80]
    ])

    regimes = [0]
    for _ in range(1, n_days):
        current_state = regimes[-1]
        next_state = np.random.choice([0, 1, 2], p=trans_matrix[current_state])
        regimes.append(next_state)

    prices = [start_price]
    highs, lows, opens, volumes = [], [], [], []

```

```
    for i in range(n_days):
        state = regimes[i]
        mu = regime_params[state]["mu"]
        sigma = regime_params[state]["sigma"]
        ret = np.random.normal(mu, sigma)

        p_close = prices[-1] * np.exp(ret)
        prices.append(p_close)

        # Approximate OHLCV
        intra_spread = abs(np.random.normal(0, sigma * 0.7))
        p_open = prices[-2] if i > 0 else start_price
        p_high = max(p_open, p_close) * (1 + intra_spread)
        p_low = min(p_open, p_close) * (1 - intra_spread)
        volume = int(np.random.lognormal(16.5, 0.4) * (1.5 if state == 2 else 1.0))

        opens.append(p_open)
        highs.append(p_high)
        lows.append(p_low)
        volumes.append(volume)

    df = pd.DataFrame({
        "Date": dates,
        "Open": opens,
        "High": highs,
        "Low": lows,
        "Close": prices[1:],
        "Volume": volumes
    }).set_index("Date")

    df["Adj Close"] = df["Close"]
    return df

def fetch_market_data(
    ticker: str = "AAPL",
    period: str = "2y",
    fallback_if_offline: bool = True
) -> Tuple[pd.DataFrame, bool]:
    """
    Fetches real-time market data from Yahoo Finance.
    Falls back cleanly to synthetic data if internet is unavailable.
    Returns: (df, is_real_data)
    """
    try:
        import yfinance as yf
        data = yf.download(ticker, period=period, progress=False)
        if data is not None and len(data) >= 60:
            if isinstance(data.columns, pd.MultiIndex):
                data.columns = [col[0] for col in data.columns]
            data = data.dropna()
            # Standardize column names
            data = data[["Open", "High", "Low", "Close", "Volume"]]
            data["Adj Close"] = data["Close"]
            return data, True
    except Exception as e:
        pass

    if fallback_if_offline:
        return generate_synthetic_market_data(ticker=ticker), False
    raise RuntimeError(f"Could not retrieve market data for {ticker}")
```

D.5 src/regime_detector.py

```
import numpy as np
import pandas as pd
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from typing import Tuple, Dict

STATE_LABELS = {
    0: "Bull (Expansion)",
    1: "Neutral (Consolidation)",
    2: "Bear (Correction/High Vol)"
}

STATE_COLORS = {
    0: "#00E676",  # Bright Green
    1: "#FFD600",  # Amber/Yellow
```

```
    2: "#FF1744"   # Crimson Red
}

class RegimeDetector:
    """
    Detects financial market regimes (Bull, Neutral, Bear) using
    unsupervised statistical clustering (Gaussian Mixture Model)
    or statistical quantile thresholds.
    """
    def __init__(self, n_regimes: int = 3, method: str = "gmm"):
        self.n_regimes = n_regimes
        self.method = method
        self.scaler = StandardScaler()
        self.model = None
        self.cluster_mapping = {}  # Maps raw cluster id to sorted regime [0=Bull, 1=Neutral, 2=Bear]

    def compute_indicators(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Enriches price data with statistical regime features:
        - Log Returns
        - 20-day Rolling Volatility (Annualized)
        - Moving Average Spread (SMA20 / SMA50 - 1)
        - RSI (Relative Strength Index)
        """
        data = df.copy()
        data["Log_Return"] = np.log(data["Close"] / data["Close"].shift(1))
        data["Rolling_Vol_20"] = data["Log_Return"].rolling(window=20).std() * np.sqrt(252)

        sma20 = data["Close"].rolling(window=20).mean()
        sma50 = data["Close"].rolling(window=50).mean()
        data["Trend_Spread"] = (sma20 / sma50) - 1.0

        # Simple RSI calculation
        delta = data["Close"].diff()
        gain = (delta.where(delta > 0, 0)).rolling(window=14).mean()
        loss = (-delta.where(delta < 0, 0)).rolling(window=14).mean()
        rs = gain / (loss + 1e-9)
        data["RSI"] = 100 - (100 / (1 + rs))

        # Momentum
        data["Mom_10"] = data["Close"].pct_change(10)

        data = data.dropna().copy()
        return data

    def fit_predict(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Fits unsupervised GMM on returns and volatility to discover market regimes.
        Clusters are sorted such that:
        State 0: Highest mean return (Bull)
        State 1: Intermediate / Low-drift return (Neutral)
        State 2: Lowest / Negative mean return with elevated vol (Bear)
        """
        featured_df = self.compute_indicators(df)

        feature_cols = ["Log_Return", "Rolling_Vol_20", "Trend_Spread"]
        X = featured_df[feature_cols].values
        X_scaled = self.scaler.fit_transform(X)

        if self.method == "gmm":
            self.model = GaussianMixture(
                n_components=self.n_regimes,
                covariance_type="full",
                random_state=42,
                n_init=5
            )
            raw_labels = self.model.fit_predict(X_scaled)
        else:
            # Fallback quantile rule
            vol_thresh = featured_df["Rolling_Vol_20"].quantile(0.65)
            ret_thresh = featured_df["Log_Return"].quantile(0.45)
            raw_labels = np.where(
                featured_df["Log_Return"] > ret_thresh,
                0,
                np.where(featured_df["Rolling_Vol_20"] > vol_thresh, 2, 1)
            )

        if self.method == "gmm":
            # GMM cluster ids are arbitrary: rank them by average return
```

```
            # (0 = highest return, 2 = lowest). Volatility is a clustering input,
            # not part of the ranking rule.
            cluster_means = []
            for c in range(self.n_regimes):
                mask = (raw_labels == c)
                mean_ret = featured_df.loc[mask, "Log_Return"].mean() if np.any(mask) else -999.0
                cluster_means.append((c, mean_ret))

            cluster_means.sort(key=lambda x: x[1], reverse=True)
            self.cluster_mapping = {original: sorted_idx for sorted_idx, (original, _) in enumerate(cluster_means)}
        else:
            # The quantile rule already emits meaningful labels (0 Bull, 1 Neutral, 2 Bear);
            # re-sorting them by mean return could scramble the volatility-based Bear rule.
            self.cluster_mapping = {c: c for c in range(self.n_regimes)}

        regime_series = np.array([self.cluster_mapping[c] for c in raw_labels])
        featured_df["Regime"] = regime_series
        featured_df["Regime_Name"] = featured_df["Regime"].map(STATE_LABELS)
        featured_df["Regime_Color"] = featured_df["Regime"].map(STATE_COLORS)

        return featured_df

    def get_regime_statistics(self, df: pd.DataFrame) -> pd.DataFrame:
        """Computes summary statistics for each detected regime."""
        stats = []
        for r_id in range(self.n_regimes):
            sub = df[df["Regime"] == r_id]
            if len(sub) == 0:
                continue
            ann_ret = (np.exp(sub["Log_Return"].mean() * 252) - 1) * 100
            ann_vol = sub["Log_Return"].std() * np.sqrt(252) * 100
            sharpe = (ann_ret - 3.0) / (ann_vol + 1e-6)  # assuming 3% risk-free rate
            stats.append({
                "Regime ID": r_id,
                "Regime Name": STATE_LABELS[r_id],
                "Count (Days)": len(sub),
                "Share (%)": f"{(len(sub) / len(df)) * 100:.1f}%",
                "Ann. Return (%)": f"{ann_ret:+.2f}%",
                "Ann. Volatility (%)": f"{ann_vol:.2f}%",
                "Sharpe Ratio": f"{sharpe:.2f}"
            })
        return pd.DataFrame(stats)
```

D.6 src/markov_model.py

```
import numpy as np
import pandas as pd
from typing import Tuple, List, Dict
from scipy import linalg

class MarkovChainModel:
    """
    First-Order Discrete-Time Markov Chain (DTMC) for Financial Market Regimes.
    Computes:
    - Transition Probability Matrix (P)
    - Stationary Distribution (pi) where pi * P = pi
    - Expected State Holding Times (1 / (1 - P_ii))
    - Multi-step Regime Transitions & Stochastic Path Generation
    """
    def __init__(self, n_states: int = 3, state_names: Dict[int, str] = None):
        self.n_states = n_states
        self.state_names = state_names or {0: "Bull", 1: "Neutral", 2: "Bear"}
        self.transition_counts = np.zeros((n_states, n_states), dtype=int)
        self.transition_matrix = np.zeros((n_states, n_states), dtype=float)
        self.stationary_distribution = np.zeros(n_states, dtype=float)
        self.expected_holding_times = np.zeros(n_states, dtype=float)

    def fit(self, regime_series: pd.Series) -> "MarkovChainModel":
        """
        Estimates the empirical transition probability matrix from a sequence of states.
        """
        states = regime_series.values
        self.transition_counts = np.zeros((self.n_states, self.n_states), dtype=int)

```

```
        for t in range(len(states) - 1):
            s_from = int(states[t])
            s_to = int(states[t + 1])
            if 0 <= s_from < self.n_states and 0 <= s_to < self.n_states:
                self.transition_counts[s_from, s_to] += 1

        # Row-normalize to probabilities
        self.transition_matrix = np.zeros((self.n_states, self.n_states), dtype=float)
        for i in range(self.n_states):
            row_sum = np.sum(self.transition_counts[i])
            if row_sum > 0:
                self.transition_matrix[i] = self.transition_counts[i] / row_sum
            else:
                self.transition_matrix[i, i] = 1.0  # absorbing state fallback

        self._compute_stationary_distribution()
        self._compute_holding_times()
        return self

    def _compute_stationary_distribution(self):
        """
        Solves for the stationary distribution pi:
        pi * P = pi  <=>  (P.T - I) * pi = 0, with sum(pi) = 1.
        """
        try:
            P = self.transition_matrix
            # Eigenvector of P.T corresponding to eigenvalue 1
            evals, evecs = linalg.eig(P.T)
            # Find the eigenvalue closest to 1.0
            idx = np.argmin(np.abs(evals - 1.0))
            stationary = np.real(evecs[:, idx])
            # Eigenvectors have an arbitrary sign: normalise FIRST so an all-negative
            # vector becomes positive (clipping first would zero it and give 0/0 = NaN).
            stationary = stationary / np.sum(stationary)
            stationary = np.maximum(stationary, 0)
            self.stationary_distribution = stationary / np.sum(stationary)
        except Exception:
            # Fallback uniform
            self.stationary_distribution = np.ones(self.n_states) / self.n_states

    def _compute_holding_times(self):
        """
        Expected holding time (sojourn time) in state i:
        E[T_i] = 1 / (1 - P_ii)
        """
        self.expected_holding_times = np.zeros(self.n_states)
        for i in range(self.n_states):
            p_ii = self.transition_matrix[i, i]
            if p_ii < 0.9999:
                self.expected_holding_times[i] = 1.0 / (1.0 - p_ii)
            else:
                self.expected_holding_times[i] = 999.0  # virtually absorbing

    def simulate_path(self, current_state: int, steps: int = 30) -> np.ndarray:
        """
        Generates a sequence of states using the transition probability matrix.
        """
        path = [current_state]
        state = current_state
        for _ in range(steps):
            probs = self.transition_matrix[state]
            state = np.random.choice(self.n_states, p=probs)
            path.append(state)
        return np.array(path[1:])  # return the future 'steps' states

    def get_transition_dataframe(self) -> pd.DataFrame:
        """Returns the transition matrix formatted as a clean DataFrame."""
        labels = [self.state_names.get(i, f"State {i}") for i in range(self.n_states)]
        df = pd.DataFrame(
            self.transition_matrix,
            index=[f"From {lbl}" for lbl in labels],
            columns=[f"To {lbl}" for lbl in labels]
        )
        return df

    def get_theoretical_summary(self) -> pd.DataFrame:
        """Returns key DTMC theoretical properties."""
        records = []
        for i in range(self.n_states):
            records.append({
                "Regime": self.state_names.get(i, f"State {i}"),
```

```
                "Self-Transition P(i->i)": f"{self.transition_matrix[i, i]:.3f}",
                "Expected Duration": f"{self.expected_holding_times[i]:.1f} days",
                "Long-Run Equilibrium (π)": f"{self.stationary_distribution[i] * 100:.1f}
%"
            })
        return pd.DataFrame(records)
```

D.7 src/ml_regressor.py

```
import numpy as np
import pandas as pd
from typing import Dict, Tuple, Any
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

class RegimeMLRegressor:
    """
    Regime-Conditioned Machine Learning Regressors.
    Trains specialized sub-models for Bull, Neutral, and Bear market regimes,
    benchmarked against a Global naive regressor.
    """
    def __init__(self, model_type: str = "ridge", alpha: float = 1.0):
        self.model_type = model_type
        self.alpha = alpha
        self.feature_cols = [
            "Log_Return", "Rolling_Vol_20", "Trend_Spread", "RSI", "Mom_10"
        ]
        self.regime_models: Dict[int, Any] = {}
        self.global_model: Any = None
        self.residuals_by_regime: Dict[int, np.ndarray] = {}

    def _create_base_pipeline(self) -> Pipeline:
        if self.model_type == "ridge":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("regressor", Ridge(alpha=self.alpha))
            ])
        elif self.model_type == "polynomial":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("poly", PolynomialFeatures(degree=2, include_bias=False)),
                ("regressor", Ridge(alpha=self.alpha))
            ])
        elif self.model_type == "random_forest":
            return Pipeline([
                ("scaler", StandardScaler()),
                ("regressor", RandomForestRegressor(n_estimators=75, max_depth=5,
random_state=42))
            ])
        else:
            return Pipeline([
                ("scaler", StandardScaler()),
                ("regressor", Ridge(alpha=self.alpha))
            ])

    def prepare_dataset(self, df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]:
        """
        Creates feature matrix X and target y (next-day log return: r_{t+1}).
        """
        data = df.copy()
        data["Target_Return"] = data["Log_Return"].shift(-1)
        data = data.dropna(subset=self.feature_cols + ["Target_Return", "Regime"]).copy()
        return data

    def fit(self, df: pd.DataFrame):
        """
        Fits both the Global model and Regime-Specific models.
        """
        data = self.prepare_dataset(df)
        X_all = data[self.feature_cols]
        y_all = data["Target_Return"]

        # Fit Global Baseline
        self.global_model = self._create_base_pipeline()
        self.global_model.fit(X_all, y_all)
```

```
69
70          # Fit Regime-Specific Models
71          unique_regimes = data["Regime"].unique()
72          for r in unique_regimes:
73              mask = (data["Regime"] == r)
74              X_r = data.loc[mask, self.feature_cols]
75              y_r = data.loc[mask, "Target_Return"]
76
77              if len(X_r) > 10:
78                  model_r = self._create_base_pipeline()
79                  model_r.fit(X_r, y_r)
80                  self.regime_models[r] = model_r
81
82                  # Compute empirical residual distribution for Monte Carlo sampling
83                  preds_r = model_r.predict(X_r)
84                  residuals = y_r.values - preds_r
85                  self.residuals_by_regime[r] = residuals
86              else:
87                  self.regime_models[r] = self.global_model
88                  self.residuals_by_regime[r] = np.zeros(10)
89
90          return self
91
92      def predict_next(self, current_features: np.ndarray, regime: int) -> float:
93          """Predicts expected return given features and active Markov regime."""
94          features_2d = pd.DataFrame([current_features], columns=self.feature_cols)
95          if regime in self.regime_models:
96              return float(self.regime_models[regime].predict(features_2d)[0])
97          return float(self.global_model.predict(features_2d)[0])
98
99      def evaluate(self, df: pd.DataFrame) -> Dict[str, Any]:
100         """
101         Evaluates Global Model vs Regime-Conditioned Model performance.
102         Metrics: R2, RMSE, Directional Accuracy (Sign Accuracy).
103         """
104         data = self.prepare_dataset(df)
105         X = data[self.feature_cols]
106         y_true = data["Target_Return"].values
107
108         # Global predictions
109         y_pred_global = self.global_model.predict(X)
110
111         # Regime-conditioned predictions
112         y_pred_regime = np.zeros_like(y_true)
113         for r, model in self.regime_models.items():
114             mask = (data["Regime"] == r).values
115             if np.any(mask):
116                 y_pred_regime[mask] = model.predict(X[mask])
117
118         def calc_metrics(y_t, y_p):
119             r2 = r2_score(y_t, y_p)
120             rmse = np.sqrt(mean_squared_error(y_t, y_p))
121             mae = mean_absolute_error(y_t, y_p)
122             # Directional accuracy: sign match
123             dir_acc = np.mean(np.sign(y_t) == np.sign(y_p)) * 100.0
124             return {
125                 "R2": r2,
126                 "RMSE": rmse,
127                 "MAE": mae,
128                 "Directional_Accuracy": dir_acc
129             }
130
131         global_metrics = calc_metrics(y_true, y_pred_global)
132         regime_metrics = calc_metrics(y_true, y_pred_regime)
133
134         comparison_df = pd.DataFrame([
135             {
136                 "Model Architecture": "Global Baseline (Naive ML)",
137                 "R² Score": f"{global_metrics['R2']:.4f}",
138                 "RMSE": f"{global_metrics['RMSE']:.5f}",
139                 "MAE": f"{global_metrics['MAE']:.5f}",
140                 "Directional Accuracy": f"{global_metrics['Directional_Accuracy']:.1f}%"
141             },
142             {
143                 "Model Architecture": "Markov Regime-Conditioned ML",
144                 "R² Score": f"{regime_metrics['R2']:.4f}",
145                 "RMSE": f"{regime_metrics['RMSE']:.5f}",
146                 "MAE": f"{regime_metrics['MAE']:.5f}",
147                 "Directional Accuracy": f"{regime_metrics['Directional_Accuracy']:.1f}%"
148             }
149         ])
```

```
150
151         return {
152             "comparison_df": comparison_df,
153             "global_metrics": global_metrics,
154             "regime_metrics": regime_metrics,
155             "y_true": y_true,
156             "y_pred_regime": y_pred_regime,
157             "y_pred_global": y_pred_global
158         }
```

D.8 src/monte_carlo.py

```
 1 import numpy as np
 2 import pandas as pd
 3 from typing import Dict, Any, List
 4 from .markov_model import MarkovChainModel
 5 from .ml_regressor import RegimeMLRegressor
 6
 7 class MonteCarloSimulator:
 8     """
 9     Hybrid Markov-ML Monte Carlo Engine.
10     Simulates hundreds of forward stochastic price paths by combining:
11     1. First-order Markov chain regime transitions.
12     2. Regime-conditioned ML regression drift predictions.
13     3. Empirical stochastic shocks sampled from regime residuals.
14     """
15     def __init__(
16         self,
17         markov_model: MarkovChainModel,
18         ml_model: RegimeMLRegressor,
19         n_simulations: int = 150,
20         forecast_days: int = 30
21     ):
22         self.markov_model = markov_model
23         self.ml_model = ml_model
24         self.n_simulations = n_simulations
25         self.forecast_days = forecast_days
26
27     def run_simulation(
28         self,
29         latest_price: float,
30         current_state: int,
31         latest_features: np.ndarray,
32         volatility_multiplier: float = 1.0,
33         forced_regime_shock: int = -1,  # -1 means natural Markov transitions, >=0 forces
    a regime
34         shock_duration: int = 5
35     ) -> Dict[str, Any]:
36         """
37         Runs the multi-path Monte Carlo simulation.
38         Supports dynamic interactive stress-testing (forced regime shock, volatility
    shifts).
39         """
40         simulated_paths = np.zeros((self.n_simulations, self.forecast_days + 1))
41         simulated_regimes = np.zeros((self.n_simulations, self.forecast_days + 1),
    dtype=int)
42
43         simulated_paths[:, 0] = latest_price
44         simulated_regimes[:, 0] = current_state
45
46         for sim_idx in range(self.n_simulations):
47             curr_price = latest_price
48             curr_state = current_state
49             curr_features = latest_features.copy()
50
51             for day in range(1, self.forecast_days + 1):
52                 # Check for forced stress-test shock
53                 if forced_regime_shock >= 0 and day <= shock_duration:
54                     next_state = forced_regime_shock
55                 else:
56                     # Transition via Markov Chain
57                     probs = self.markov_model.transition_matrix[curr_state]
58                     next_state = int(np.random.choice(self.markov_model.n_states,
    p=probs))
59
60                 # Expected drift from regime ML model
61                 expected_ret = self.ml_model.predict_next(curr_features, next_state)
62
```

```
                # Stochastic shock from empirical residual distribution
                resids = self.ml_model.residuals_by_regime.get(next_state, np.zeros(1))
                if len(resids) > 5:
                    shock = np.random.choice(resids) * volatility_multiplier
                else:
                    default_sigma = 0.015 if next_state == 2 else (0.010 if next_state == 1 else 0.008)
                    shock = np.random.normal(0, default_sigma) * volatility_multiplier

                sim_return = expected_ret + shock
                # Prevent extreme exponential explosion in simulations
                sim_return = np.clip(sim_return, -0.15, 0.15)

                curr_price = curr_price * np.exp(sim_return)
                simulated_paths[sim_idx, day] = curr_price
                simulated_regimes[sim_idx, day] = next_state

                # State-space feature update for next step.
                # LIMITATION: only the return (index 0) is updated; volatility, trend, RSI and
                # momentum stay frozen at their last observed values.
                curr_features[0] = sim_return  # updated return
                curr_state = next_state

        # Compute quantiles across all paths
        median_path = np.median(simulated_paths, axis=0)
        p10_path = np.percentile(simulated_paths, 10, axis=0)
        p25_path = np.percentile(simulated_paths, 25, axis=0)
        p75_path = np.percentile(simulated_paths, 75, axis=0)
        p90_path = np.percentile(simulated_paths, 90, axis=0)

        terminal_prices = simulated_paths[:, -1]
        terminal_returns = (terminal_prices - latest_price) / latest_price * 100.0

        # Risk metrics
        var_95 = np.percentile(terminal_returns, 5)  # 95% 1-tail VaR
        expected_terminal_return = np.median(terminal_returns)
        prob_positive = np.mean(terminal_returns > 0) * 100.0

        return {
            "paths": simulated_paths,
            "regimes": simulated_regimes,
            "median": median_path,
            "p10": p10_path,
            "p25": p25_path,
            "p75": p75_path,
            "p90": p90_path,
            "terminal_prices": terminal_prices,
            "terminal_returns": terminal_returns,
            "expected_terminal_return": expected_terminal_return,
            "var_95": var_95,
            "prob_positive": prob_positive
        }
```

D.9 src/visualizations.py

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from typing import Dict, Any, List

# Aesthetic Dark Fintech Theme Palette
PALETTE = {
    "bg": "#0E1117",
    "card_bg": "#161B22",
    "text": "#E6EDF3",
    "text_muted": "#8B949E",
    "bull": "#00E676",      # Bright neon green
    "neutral": "#FFD600",   # Electric amber
    "bear": "#FF1744",      # High-vis red
    "accent_cyan": "#00E5FF",
    "accent_purple": "#B388FF",
    "grid": "#21262D"
}

def create_candlestick_regime_chart(df: pd.DataFrame, ticker: str = "ASSET") -> go.Figure:
    """
```

```
    Creates an interactive financial chart showing price action
    color-coded by the detected Markov regime.
    """
    fig = make_subplots(
        rows=2, cols=1,
        shared_xaxes=True,
        vertical_spacing=0.04,
        row_heights=[0.75, 0.25],
        subplot_titles=[f"<b>{ticker}</b> Price Action & Detected Markov Regimes",
"Trading Volume & Regime Intensity"]
    )

    # 1. Price Candlesticks / Close line
    fig.add_trace(
        go.Candlestick(
            x=df.index,
            open=df["Open"],
            high=df["High"],
            low=df["Low"],
            close=df["Close"],
            name="OHLC Price",
            increasing_line_color="#26A69A",
            decreasing_line_color="#EF5350"
        ),
        row=1, col=1
    )

    # Add 20-day and 50-day SMA if present
    if "Rolling_Vol_20" in df.columns:
        sma20 = df["Close"].rolling(20).mean()
        fig.add_trace(
            go.Scatter(
                x=df.index, y=sma20,
                name="20 SMA",
                line=dict(color="#29B6F6", width=1.5, dash="dot")
            ),
            row=1, col=1
        )

    # 2. Volume Bar Chart
    colors = [
        PALETTE["bull"] if r == 0 else (PALETTE["neutral"] if r == 1 else PALETTE["bear"])
        for r in df["Regime"]
    ]
    fig.add_trace(
        go.Bar(
            x=df.index,
            y=df["Volume"],
            marker_color=colors,
            opacity=0.7,
            name="Regime Volume"
        ),
        row=2, col=1
    )

    # 3. Add background color spans for regimes (consecutive blocks)
    regimes = df["Regime"].values
    dates = df.index
    n = len(df)

    start_idx = 0
    while start_idx < n:
        curr_r = regimes[start_idx]
        end_idx = start_idx
        while end_idx + 1 < n and regimes[end_idx + 1] == curr_r:
            end_idx += 1

        color = "rgba(0, 230, 118, 0.12)" if curr_r == 0 else (
            "rgba(255, 214, 0, 0.10)" if curr_r == 1 else "rgba(255, 23, 68, 0.15)"
        )
        fig.add_vrect(
            x0=dates[start_idx],
            x1=dates[end_idx],
            fillcolor=color,
            layer="below",
            line_width=0,
            row=1, col=1
        )
        start_idx = end_idx + 1

    fig.update_layout(
```

```
        template="plotly_dark",
        plot_bgcolor=PALETTE["bg"],
        paper_bgcolor=PALETTE["bg"],
        xaxis_rangeslider_visible=False,
        height=580,
        margin=dict(l=40, r=40, t=50, b=30),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )
    fig.update_xaxes(showgrid=True, gridcolor=PALETTE["grid"])
    fig.update_yaxes(showgrid=True, gridcolor=PALETTE["grid"])
    return fig


def create_markov_network_diagram(transition_matrix: np.ndarray, stationary_dist:
np.ndarray) -> go.Figure:
    """
    Creates an interactive graphical state transition network diagram
    where nodes represent Bull, Neutral, Bear states and directed arrows
    represent transition probabilities.
    """
    fig = go.Figure()

    # Fixed triangle geometry for the 3 states
    coords = {
        0: (0.20, 0.75),   # Bull (top left)
        1: (0.80, 0.75),   # Neutral (top right)
        2: (0.50, 0.20)    # Bear (bottom center)
    }
    state_names = ["Bull (Expansion)", "Neutral (Consolidation)", "Bear (Correction)"]
    node_colors = [PALETTE["bull"], PALETTE["neutral"], PALETTE["bear"]]

    # 1. Draw Transition Arrows / Edges
    for i in range(3):
        for j in range(3):
            prob = transition_matrix[i, j]
            if prob < 0.02:
                continue

            x0, y0 = coords[i]
            x1, y1 = coords[j]

            if i == j:
                # Self-loop indicator above/below node
                offset_y = 0.11 if y0 > 0.5 else -0.11
                fig.add_annotation(
                    x=x0, y=y0 + offset_y,
                    text=f"<b>P({i}→{i})<br>{prob*100:.1f}%</b>",
                    showarrow=False,
                    font=dict(color=node_colors[i], size=11),
                    bgcolor="rgba(22, 27, 34, 0.8)",
                    bordercolor=node_colors[i],
                    borderwidth=1,
                    borderpad=3
                )
            else:
                # Directed edge with midpoint probability label
                # Slightly curve the midpoints so opposing arrows don't collide
                mx = (x0 + x1) / 2.0
                my = (y0 + y1) / 2.0

                # Perpendicular offset for curve
                dx = x1 - x0
                dy = y1 - y0
                curve_strength = 0.08
                cx = mx - dy * curve_strength
                cy = my + dx * curve_strength

                # Curved path
                path_str = f"M {x0},{y0} Q {cx},{cy} {x1},{y1}"
                edge_color = node_colors[i]

                fig.add_shape(
                    type="path",
                    path=path_str,
                    line=dict(color=edge_color, width=max(1.5, prob * 6)),
                    opacity=0.65
                )

                # Midpoint arrow label
                fig.add_annotation(
                    x=cx, y=cy,
```

```
                    text=f"<b>{prob*100:.1f}%</b>",
                    showarrow=False,
                    font=dict(color=PALETTE["text"], size=10),
                    bgcolor="rgba(22, 27, 34, 0.9)",
                    bordercolor=edge_color,
                    borderwidth=1,
                    borderpad=2
                )

    # 2. Draw State Nodes (sized by stationary distribution)
    node_x = [coords[i][0] for i in range(3)]
    node_y = [coords[i][1] for i in range(3)]
    node_sizes = [max(45, stationary_dist[i] * 120) for i in range(3)]
    hover_texts = [
        f"<b>{state_names[i]}</b><br>Equilibrium Share (π): {stationary_dist[i]*100:.1f}%<br>Self-Persistence: {transition_matrix[i,i]*100:.1f}%"
        for i in range(3)
    ]

    fig.add_trace(
        go.Scatter(
            x=node_x, y=node_y,
            mode="markers+text",
            marker=dict(
                size=node_sizes,
                color=node_colors,
                line=dict(color="#FFFFFF", width=2),
                opacity=0.95
            ),
            text=[f"<b>{state_names[i].split()[0]}</b><br>π={stationary_dist[i]*100:.1f}%" for i in range(3)],
            textposition="middle center",
            textfont=dict(color="#000000", size=11, family="Arial Black"),
            hoverinfo="text",
            hovertext=hover_texts,
            name="Regime Nodes"
        )
    )

    fig.update_layout(
        title="<b>Markov State Transition Graph (Self-Loops & Equilibrium π)</b>",
        template="plotly_dark",
        plot_bgcolor=PALETTE["card_bg"],
        paper_bgcolor=PALETTE["bg"],
        xaxis=dict(showgrid=False, zeroline=False, showticklabels=False, range=[0.05, 0.95]),
        yaxis=dict(showgrid=False, zeroline=False, showticklabels=False, range=[0.05, 0.95]),
        height=420,
        margin=dict(l=20, r=20, t=50, b=20),
        showlegend=False
    )
    return fig


def create_monte_carlo_fan_chart(
    historical_dates: pd.DatetimeIndex,
    historical_prices: pd.Series,
    sim_results: Dict[str, Any],
    ticker: str = "ASSET"
) -> go.Figure:
    """
    Creates the Monte Carlo probability fan chart
    with glowing median trajectory and 50% & 80% confidence bands.
    """
    fig = go.Figure()

    # Lookback history (last 60 trading days)
    hist_sub = historical_prices.iloc[-60:]
    last_date = hist_sub.index[-1]

    # Future dates
    forecast_days = sim_results["paths"].shape[1] - 1
    future_dates = pd.date_range(start=last_date, periods=forecast_days + 1, freq="B")

    # 1. Historical actual price line
    fig.add_trace(
        go.Scatter(
            x=hist_sub.index,
            y=hist_sub.values,
            mode="lines",
```

```
            name="Historical Actual",
            line=dict(color="#FFFFFF", width=2.5)
        )
    )

    # 2. Individual Simulated Stochastic Paths (translucent neon lines)
    sample_indices = np.random.choice(sim_results["paths"].shape[0], size=min(60, sim_results["paths"].shape[0]), replace=False)
    for idx in sample_indices:
        fig.add_trace(
            go.Scatter(
                x=future_dates,
                y=sim_results["paths"][idx],
                mode="lines",
                line=dict(color="rgba(0, 229, 255, 0.08)", width=1),
                showlegend=False,
                hoverinfo="skip"
            )
        )

    # 3. 80% Confidence Interval Band (10th to 90th percentile)
    fig.add_trace(
        go.Scatter(
            x=future_dates,
            y=sim_results["p90"],
            mode="lines",
            line=dict(width=0),
            showlegend=False,
            hoverinfo="skip"
        )
    )
    fig.add_trace(
        go.Scatter(
            x=future_dates,
            y=sim_results["p10"],
            mode="lines",
            line=dict(width=0),
            fill="tonexty",
            fillcolor="rgba(0, 229, 255, 0.12)",
            name="80% Probability Interval",
            hoverinfo="skip"
        )
    )

    # 4. 50% Confidence Interval Band (25th to 75th percentile)
    fig.add_trace(
        go.Scatter(
            x=future_dates,
            y=sim_results["p75"],
            mode="lines",
            line=dict(width=0),
            showlegend=False,
            hoverinfo="skip"
        )
    )
    fig.add_trace(
        go.Scatter(
            x=future_dates,
            y=sim_results["p25"],
            mode="lines",
            line=dict(width=0),
            fill="tonexty",
            fillcolor="rgba(179, 136, 255, 0.22)",
            name="50% Interquartile Range",
            hoverinfo="skip"
        )
    )

    # 5. Median Trajectory (Glowing gold)
    fig.add_trace(
        go.Scatter(
            x=future_dates,
            y=sim_results["median"],
            mode="lines+markers",
            marker=dict(size=4),
            name="Median Forecast (Markov + ML)",
            line=dict(color="#FFD600", width=3, dash="solid")
        )
    )

    # Vertical split line
```

```
    fig.add_vline(
        x=last_date.timestamp() * 1000,
        line_width=1.5,
        line_dash="dash",
        line_color="#8B949E",
        annotation_text="Today (Forecast Start)",
        annotation_position="top left",
        annotation_font=dict(color="#8B949E", size=10)
    )

    fig.update_layout(
        title=f"<b>Monte Carlo Multi-Regime Prediction Cone ({forecast_days} Trading Days Forward)</b>",
        template="plotly_dark",
        plot_bgcolor=PALETTE["bg"],
        paper_bgcolor=PALETTE["bg"],
        height=520,
        margin=dict(l=40, r=40, t=60, b=30),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )
    fig.update_xaxes(showgrid=True, gridcolor=PALETTE["grid"])
    fig.update_yaxes(showgrid=True, gridcolor=PALETTE["grid"])
    return fig


def create_terminal_distribution_chart(sim_results: Dict[str, Any]) -> go.Figure:
    """Creates a histogram of terminal simulated returns with VaR and median metrics."""
    rets = sim_results["terminal_returns"]
    median_ret = sim_results["expected_terminal_return"]
    var_95 = sim_results["var_95"]

    fig = go.Figure()
    fig.add_trace(
        go.Histogram(
            x=rets,
            nbinsx=35,
            marker_color=PALETTE["accent_cyan"],
            opacity=0.75,
            name="Simulated Return Density"
        )
    )

    # Median line
    fig.add_vline(
        x=median_ret,
        line_color=PALETTE["bull"],
        line_width=2.5,
        line_dash="dash",
        annotation_text=f"Median: {median_ret:+.1f}%",
        annotation_position="top right"
    )

    # 95% VaR line
    fig.add_vline(
        x=var_95,
        line_color=PALETTE["bear"],
        line_width=2.5,
        line_dash="dot",
        annotation_text=f"95% VaR: {var_95:+.1f}%",
        annotation_position="top left"
    )

    fig.update_layout(
        title=f"<b>Distribution of Terminal Returns Across {len(sim_results['terminal_returns'])} Stochastic Paths</b>",
        template="plotly_dark",
        plot_bgcolor=PALETTE["card_bg"],
        paper_bgcolor=PALETTE["bg"],
        xaxis_title="Forward Cumulative Return (%)",
        yaxis_title="Path Frequency",
        height=380,
        margin=dict(l=40, r=40, t=50, b=30),
        showlegend=False
    )
    fig.update_xaxes(showgrid=True, gridcolor=PALETTE["grid"])
    fig.update_yaxes(showgrid=True, gridcolor=PALETTE["grid"])
    return fig


def create_model_comparison_bar(comparison_df: pd.DataFrame) -> go.Figure:
    """Creates side-by-side metric comparison bar chart between baseline and Markov-ML."""
```

```
    fig = go.Figure()

    models = comparison_df["Model Architecture"].tolist()
    r2_vals = [float(x) for x in comparison_df["R² Score"]]
    dir_acc = [float(x.replace("%", "")) for x in comparison_df["Directional Accuracy"]]

    fig.add_trace(
        go.Bar(
            x=models,
            y=dir_acc,
            name="Directional Accuracy (%)",
            marker_color=[PALETTE["accent_purple"], PALETTE["bull"]],
            text=[f"{v:.1f}%" for v in dir_acc],
            textposition="auto"
        )
    )

    fig.update_layout(
        title="<b>Predictive Edge: Baseline ML vs Markov-Conditioned Model</b>",
        template="plotly_dark",
        plot_bgcolor=PALETTE["card_bg"],
        paper_bgcolor=PALETTE["bg"],
        yaxis=dict(title="Directional Sign Accuracy (%)", range=[35, 75]),
        height=380,
        margin=dict(l=40, r=40, t=50, b=30),
        showlegend=False
    )
    return fig


def generate_matplotlib_report_figure(
    df: pd.DataFrame,
    transition_matrix: np.ndarray,
    stationary_dist: np.ndarray,
    sim_results: Dict[str, Any],
    ticker: str = "ASSET"
):
    """
    Generates a publication-grade, 4-panel 300-DPI Matplotlib figure
    ideal for thesis reports, presentation slides, or LaTeX paper exports.
    """
    import matplotlib.pyplot as plt
    import matplotlib.dates as mdates

    fig, axes = plt.subplots(2, 2, figsize=(14, 9), dpi=300)
    fig.patch.set_facecolor("#FFFFFF")
    plt.subplots_adjust(hspace=0.32, wspace=0.25)

    # 1. Price Action with Regime Color-Coding
    ax1 = axes[0, 0]
    ax1.plot(df.index, df["Close"], color="#1A237E", linewidth=1.5, label="Close Price")
    if "Rolling_Vol_20" in df.columns:
        sma20 = df["Close"].rolling(20).mean()
        ax1.plot(df.index, sma20, color="#00838F", linestyle="--", linewidth=1.0,
label="20 SMA")

    regimes = df["Regime"].values
    dates = df.index
    for i in range(len(df) - 1):
        c = "#C8E6C9" if regimes[i] == 0 else ("#FFF9C4" if regimes[i] == 1 else
"#FFCDD2")
        ax1.axvspan(dates[i], dates[i+1], color=c, alpha=0.35, linewidth=0)

    ax1.set_title(f"{ticker} Historical Trajectory & Classified Regimes", fontsize=11,
fontweight="bold")
    ax1.set_ylabel("Price ($)", fontsize=10)
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", fontsize=8)

    # 2. Transition Matrix Heatmap
    ax2 = axes[0, 1]
    cax = ax2.imshow(transition_matrix, cmap="Blues", vmin=0, vmax=1)
    labels = ["Bull", "Neutral", "Bear"]
    ax2.set_xticks(range(3))
    ax2.set_yticks(range(3))
    ax2.set_xticklabels([f"To {l}" for l in labels], fontsize=9)
    ax2.set_yticklabels([f"From {l}" for l in labels], fontsize=9)
    ax2.set_title(r"DTMC Transition Probability Matrix $P_{ij}$", fontsize=11,
fontweight="bold")

    for i in range(3):
```

```
        for j in range(3):
            val = transition_matrix[i, j]
            color = "white" if val > 0.5 else "black"
            ax2.text(j, i, f"{val:.3f}", ha="center", va="center", color=color, fontweight="bold", fontsize=10)
    fig.colorbar(cax, ax=ax2, shrink=0.75)

    # 3. Forward Monte Carlo Fan Cone
    ax3 = axes[1, 0]
    hist_sub = df["Close"].iloc[-45:]
    forecast_days = sim_results["paths"].shape[1] - 1
    future_x = np.arange(len(hist_sub), len(hist_sub) + forecast_days + 1)
    past_x = np.arange(len(hist_sub))

    ax3.plot(past_x, hist_sub.values, color="black", linewidth=1.5, label="Observed Price")

    # 80% and 50% bands
    ax3.fill_between(future_x, sim_results["p10"], sim_results["p90"], color="#80DEEA", alpha=0.4, label="80% Probability Interval")
    ax3.fill_between(future_x, sim_results["p25"], sim_results["p75"], color="#B39DDB", alpha=0.5, label="50% Interquartile Range")
    ax3.plot(future_x, sim_results["median"], color="#FF6F00", linewidth=2.0, label="Median Forecast")

    # A few random individual trajectories
    sample_indices = np.random.choice(sim_results["paths"].shape[0], size=min(15, sim_results["paths"].shape[0]), replace=False)
    for s_idx in sample_indices:
        ax3.plot(future_x, sim_results["paths"][s_idx], color="#00ACC1", alpha=0.15, linewidth=0.8)

    ax3.set_title(f"Monte Carlo Stochastic Fan ({forecast_days} Trading Days Forward)", fontsize=11, fontweight="bold")
    ax3.set_ylabel("Price ($)", fontsize=10)
    ax3.set_xlabel("Relative Trading Days", fontsize=10)
    ax3.grid(True, linestyle=":", alpha=0.6)
    ax3.legend(loc="upper left", fontsize=8)

    # 4. Long-Run Stationary Equilibrium π
    ax4 = axes[1, 1]
    bar_colors = ["#43A047", "#FBC02D", "#E53935"]
    bars = ax4.bar(labels, stationary_dist * 100, color=bar_colors, width=0.55, edgecolor="black", linewidth=0.8)
    ax4.set_title(r"Long-Run Stationary Equilibrium ($\boldsymbol{\pi}$)", fontsize=11, fontweight="bold")
    ax4.set_ylabel("Equilibrium Time Share (%)", fontsize=10)
    ax4.grid(axis="y", linestyle=":", alpha=0.6)
    for bar in bars:
        h = bar.get_height()
        ax4.text(bar.get_x() + bar.get_width()/2., h + 1.0, f"{h:.1f}%", ha="center", va="bottom", fontsize=10, fontweight="bold")
    ax4.set_ylim(0, max(stationary_dist * 100) + 15)

    return fig
```

D.10 app.py

```
import streamlit as st
import numpy as np
import pandas as pd
import datetime
import io

from src.data_loader import fetch_market_data, generate_synthetic_market_data
from src.regime_detector import RegimeDetector, STATE_LABELS, STATE_COLORS
from src.markov_model import MarkovChainModel
from src.ml_regressor import RegimeMLRegressor
from src.monte_carlo import MonteCarloSimulator
from src.visualizations import (
    create_candlestick_regime_chart,
    create_markov_network_diagram,
    create_monte_carlo_fan_chart,
    create_terminal_distribution_chart,
    create_model_comparison_bar,
    generate_matplotlib_report_figure,
    PALETTE
)
```

```
21
22  # Page configuration
23  st.set_page_config(
24      page_title="QuantRegime AI | Markov-ML Asset Forecaster",
25      page_icon="📈",
26      layout="wide",
27      initial_sidebar_state="expanded"
28  )
29
30  # Custom High-Impact CSS Styling
31  st.markdown("""
32  <style>
33      /* Global Page Styling */
34      .stApp {
35          background-color: #0B0E14;
36          color: #E6EDF3;
37          font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica,
    Arial, sans-serif;
38      }
39
40      /* Metric Card Styling */
41      div[data-testid="stMetric"] {
42          background: linear-gradient(135deg, #161B22 0%, #1F242C 100%);
43          border: 1px solid #30363D;
44          border-radius: 12px;
45          padding: 14px 18px;
46          box-shadow: 0 4px 15px rgba(0, 0, 0, 0.4);
47      }
48      div[data-testid="stMetric"]:hover {
49          border-color: #00E5FF;
50          transform: translateY(-2px);
51          transition: all 0.2s ease-in-out;
52      }
53      div[data-testid="stMetricLabel"] {
54          color: #8B949E !important;
55          font-size: 0.85rem !important;
56          font-weight: 600;
57          text-transform: uppercase;
58          letter-spacing: 0.5px;
59      }
60      div[data-testid="stMetricValue"] {
61          color: #FFFFFF !important;
62          font-size: 1.8rem !important;
63          font-weight: 700;
64      }
65
66      /* Header Gradient Banner */
67      .hero-banner {
68          background: linear-gradient(90deg, rgba(0, 229, 255, 0.12) 0%, rgba(179, 136, 255,
    0.15) 100%);
69          border: 1px solid rgba(0, 229, 255, 0.3);
70          border-radius: 14px;
71          padding: 22px 28px;
72          margin-bottom: 24px;
73      }
74      .hero-title {
75          font-size: 2.2rem;
76          font-weight: 800;
77          background: linear-gradient(120deg, #00E5FF, #B388FF, #00E676);
78          -webkit-background-clip: text;
79          -webkit-text-fill-color: transparent;
80          margin-bottom: 6px;
81      }
82      .hero-sub {
83          color: #8B949E;
84          font-size: 1.05rem;
85          margin: 0;
86      }
87
88      /* Custom Badge */
89      .regime-badge {
90          display: inline-block;
91          padding: 6px 14px;
92          border-radius: 20px;
93          font-weight: 700;
94          font-size: 0.9rem;
95          letter-spacing: 0.5px;
96      }
97  </style>
98  """, unsafe_allow_html=True)
99
```

```
# ----------------- SIDEBAR CONTROLS -----------------
with st.sidebar:
    st.image("https://img.icons8.com/isometric-line/100/line-chart.png", width=64)
    st.title("Control Deck")
    st.caption("Markov Chains & ML Regression")

    st.markdown("---")
    st.subheader("1. Asset & Historical Data")
    preset_choice = st.selectbox(
        "Select Market Asset",
        ["SPY (S&P 500)", "NVDA (Nvidia)", "AAPL (Apple)", "TSLA (Tesla)", "BTC-USD (Bitcoin)", "Custom Ticker", "Synthetic Demo (Offline)"]
    )

    if preset_choice == "Custom Ticker":
        ticker = st.text_input("Enter Yahoo Finance Ticker", value="MSFT").upper()
    elif preset_choice == "Synthetic Demo (Offline)":
        ticker = "SYNTH_SPY"
    else:
        ticker = preset_choice.split(" ")[0]

    period = st.selectbox("Historical Lookback", ["1y", "2y", "3y", "5y"], index=1)

    st.markdown("---")
    st.subheader("2. Markov & ML Setup")
    detector_method = st.selectbox("Regime Classifier", ["gmm", "quantile"], format_func=lambda x: "Gaussian Mixture Model (GMM)" if x == "gmm" else "Quantile Rule")
    regressor_type = st.selectbox("ML Regressor", ["ridge", "polynomial", "random_forest"], format_func=lambda x: {"ridge": "Ridge Regression (L2)", "polynomial": "Polynomial Features (deg=2)", "random_forest": "Random Forest Ensemble"}[x])

    st.markdown("---")
    st.subheader("3. Monte Carlo Sandbox")
    forecast_days = st.slider("Forecast Horizon (Trading Days)", min_value=10, max_value=60, value=30, step=5)
    n_simulations = st.slider("Simulated Stochastic Paths", min_value=50, max_value=300, value=150, step=25)
    vol_multiplier = st.slider("Volatility Stress Multiplier", min_value=0.5, max_value=2.5, value=1.0, step=0.1)

    shock_mode = st.selectbox(
        "Simulate Shock (Stress Test)",
        ["None (Natural Markov Flow)", "Black Swan: Sudden Bear Crash", "Surge: Sudden Bull Explosion"]
    )
    forced_shock = -1
    if "Bear Crash" in shock_mode:
        forced_shock = 2
    elif "Bull Explosion" in shock_mode:
        forced_shock = 0

# ----------------- DATA PIPELINE -----------------
@st.cache_data(show_spinner=False, ttl=3600)
def load_and_process_pipeline(ticker_sym: str, period_str: str, method: str, reg_type: str):
    if ticker_sym == "SYNTH_SPY":
        raw_df = generate_synthetic_market_data()
        is_live = False
    else:
        raw_df, is_live = fetch_market_data(ticker=ticker_sym, period=period_str, fallback_if_offline=True)

    detector = RegimeDetector(n_regimes=3, method=method)
    classified_df = detector.fit_predict(raw_df)
    regime_stats = detector.get_regime_statistics(classified_df)

    markov = MarkovChainModel(n_states=3, state_names=STATE_LABELS)
    markov.fit(classified_df["Regime"])

    # Final model (fitted on all rows) is used for the Monte Carlo forecast.
    ml = RegimeMLRegressor(model_type=reg_type)
    ml.fit(classified_df)

    # Honest evaluation: chronological 70/30 split. Models are fitted on the first
    # 70% of rows only and scored on the last 30%. (Regime labels still come from a
    # GMM fitted on the full history, so this is a descriptive hold-out check, not a
    # walk-forward forecast test.)
    split = int(len(classified_df) * 0.7)
    ml_holdout = RegimeMLRegressor(model_type=reg_type)
    ml_holdout.fit(classified_df.iloc[:split])
    ml_eval = ml_holdout.evaluate(classified_df.iloc[split:])
```

```
171
172     return {
173         "raw_df": raw_df,
174         "is_live": is_live,
175         "classified_df": classified_df,
176         "regime_stats": regime_stats,
177         "markov": markov,
178         "ml": ml,
179         "ml_eval": ml_eval
180     }
181
182 with st.spinner("Crunching historical market regimes & training ML models..."):
183     pipeline = load_and_process_pipeline(ticker, period, detector_method, regressor_type)
184
185 df = pipeline["classified_df"]
186 markov = pipeline["markov"]
187 ml = pipeline["ml"]
188 eval_res = pipeline["ml_eval"]
189 current_regime = int(df["Regime"].iloc[-1])
190 current_price = float(df["Close"].iloc[-1])
191 prev_price = float(df["Close"].iloc[-2])
192 day_change = ((current_price - prev_price) / prev_price) * 100
193
194 # Monte Carlo Execution
195 mc = MonteCarloSimulator(markov_model=markov, ml_model=ml, n_simulations=n_simulations,
    forecast_days=forecast_days)
196 latest_features = df[ml.feature_cols].iloc[-1].values
197 sim_results = mc.run_simulation(
198     latest_price=current_price,
199     current_state=current_regime,
200     latest_features=latest_features,
201     volatility_multiplier=vol_multiplier,
202     forced_regime_shock=forced_shock,
203     shock_duration=5
204 )
205
206 # ----------------- MAIN UI -----------------
207 st.markdown(f"""
208 <div class="hero-banner">
209     <div class="hero-title">QUANTREGIME AI</div>
210     <div class="hero-sub">
211         Discrete Markov Chains & Conditioned Machine Learning Regression
212         {' &bull; <span style="color:#00E676;">● Live Data</span>' if pipeline['is_live']
    else ' &bull; <span style="color:#FFD600;">● High-Fidelity Synthetic Fallback</span>'}
213     </div>
214 </div>
215 """, unsafe_allow_html=True)
216
217 # Top KPI Row
218 col1, col2, col3, col4, col5 = st.columns(5)
219 with col1:
220     st.metric(
221         label=f"{ticker} Latest Close",
222         value=f"${current_price:,.2f}",
223         delta=f"{day_change:+.2f}% (1D)"
224     )
225
226 with col2:
227     regime_name = STATE_LABELS[current_regime].split(" ")[0]
228     color_map = {0: "#00E676", 1: "#FFD600", 2: "#FF1744"}
229     st.markdown(f"""
230     <div style="background: linear-gradient(135deg, #161B22 0%, #1F242C 100%); border: 1px
    solid #30363D; border-radius: 12px; padding: 14px 18px; box-shadow: 0 4px 15px
    rgba(0,0,0,0.4);">
231         <div style="color: #8B949E; font-size: 0.85rem; font-weight: 600; text-transform:
    uppercase;">Active Markov State</div>
232         <div style="color: {color_map[current_regime]}; font-size: 1.6rem; font-weight:
    800; margin-top: 4px;">
233             ● {regime_name}
234         </div>
235     </div>
236     """, unsafe_allow_html=True)
237
238 with col3:
239     median_terminal = sim_results["median"][-1]
240     expected_pct = sim_results["expected_terminal_return"]
241     st.metric(
242         label=f"{forecast_days}-Day Median Target",
243         value=f"${median_terminal:,.2f}",
244         delta=f"{expected_pct:+.2f}%"
245     )
```

```
246
247 with col4:
248     regime_dir_acc = eval_res["regime_metrics"]["Directional_Accuracy"]
249     global_dir_acc = eval_res["global_metrics"]["Directional_Accuracy"]
250     edge = regime_dir_acc - global_dir_acc
251     st.metric(
252         label="Hold-out Directional Accuracy",
253         value=f"{regime_dir_acc:.1f}%",
254         delta=f"{edge:+.1f} pts vs global model"
255     )
256
257 with col5:
258     var95 = sim_results["var_95"]
259     st.metric(
260         label="95% VaR (5th-pct simulated return)",
261         value=f"{var95:.1f}%",
262         delta="Left-tail return over horizon",
263         delta_color="off"
264     )
265
266 st.write("")
267
268 # Main Tabs
269 tab1, tab2, tab3, tab4, tab5 = st.tabs([
270     "🎯 Stochastic Forecast Fan",
271     "🔄 Markov Chain Engine",
272     "📊 Regime-Partitioned Price Action",
273     "🧠 ML Architecture & Hold-out Check",
274     "🎓 Faculty Viva Defense Guide"
275 ])
276
277 # ----------------- TAB 1: MONTE CARLO STOCHASTIC FAN -----------------
278 with tab1:
279     st.plotly_chart(
280         create_monte_carlo_fan_chart(df.index, df["Close"], sim_results, ticker=ticker),
281         use_container_width=True
282     )
283
284     col_a, col_b = st.columns([1.2, 0.8])
285     with col_a:
286         st.plotly_chart(create_terminal_distribution_chart(sim_results),
    use_container_width=True)
287     with col_b:
288         st.markdown(f"""
289         <div style="background: #161B22; border: 1px solid #30363D; border-radius: 12px;
    padding: 20px; height: 380px;">
290             <h4 style="color: #00E5FF; margin-top: 0;">🎲 Monte Carlo Risk Breakdown</h4>
291             <p style="color: #8B949E; font-size: 0.95rem;">
292                 Based on <b>{n_simulations} simulated stochastic trajectories</b> across
    <b>{forecast_days} forward trading days</b>, integrating Markov state switching with ML
    regression residuals.
293             </p>
294             <hr style="border-color: #30363D;">
295             <div style="display: flex; justify-content: space-between; margin-bottom:
    10px;">
296                 <span style="color: #E6EDF3;">Probability of Positive Return:</span>
297                 <span style="color: #00E676; font-weight:
    bold;">{sim_results['prob_positive']:.1f}%</span>
298             </div>
299             <div style="display: flex; justify-content: space-between; margin-bottom:
    10px;">
300                 <span style="color: #E6EDF3;">Best-Case 90th Percentile:</span>
301                 <span style="color: #00E5FF; font-weight: bold;">${sim_results['p90'][-
    1]:.2f}</span>
302             </div>
303             <div style="display: flex; justify-content: space-between; margin-bottom:
    10px;">
304                 <span style="color: #E6EDF3;">Worst-Case 10th Percentile:</span>
305                 <span style="color: #FF1744; font-weight: bold;">${sim_results['p10'][-
    1]:.2f}</span>
306             </div>
307             <div style="display: flex; justify-content: space-between; margin-bottom:
    10px;">
308                 <span style="color: #E6EDF3;">Simulated Volatility Multiplier:</span>
309                 <span style="color: #FFD600; font-weight: bold;">{vol_multiplier}x</span>
310             </div>
311             <div style="margin-top: 15px; background: rgba(0, 229, 255, 0.08); padding:
    10px; border-radius: 8px; font-size: 0.85rem; color: #8B949E;">
312                 💡 <i>Tip for Demo:</i> Use the sidebar <b>"Simulate Shock"</b> slider to
    trigger a Black Swan crash in real-time.
313             </div>
```

```
        </div>
        """, unsafe_allow_html=True)

# ----------------- TAB 2: MARKOV CHAIN ENGINE -----------------
with tab2:
    col_m1, col_m2 = st.columns([1.1, 0.9])
    with col_m1:
        st.plotly_chart(
            create_markov_network_diagram(markov.transition_matrix,
markov.stationary_distribution),
            use_container_width=True
        )
    with col_m2:
        st.markdown("#### 📐 Empirical Transition Matrix $P$")
        st.caption(r"Rows sum to 1.0; each cell $P_{ij} = P(S_{t+1}=j \mid S_t=i)$")

st.dataframe(markov.get_transition_dataframe().style.format("{:.3f}").background_gradient(
cmap="Blues", axis=1), use_container_width=True)

        st.markdown("#### ⚖️ DTMC Ergodic & Equilibrium Properties")
        st.dataframe(markov.get_theoretical_summary(), use_container_width=True)

    with st.expander("📚 Faculty Explainer: Mathematical Formulation of Discrete Markov
Chains"):
        st.markdown(r"""
        ### 1. The Markov Property
        A stochastic process $\{X_t, t \ge 0\}$ satisfies the First-Order Markov Property
if the conditional probability distribution of future states depends solely upon the
present state:
        $$P(X_{t+1} = s_{t+1} \mid X_t = s_t, X_{t-1} = s_{t-1}, \dots, X_0 = s_0) =
P(X_{t+1} = s_{t+1} \mid X_t = s_t)$$

        ### 2. Maximum Likelihood Estimation of $P_{ij}$
        The transition probabilities are calculated via empirical frequencies:
        $$p_{ij} = \frac{N_{ij}}{\sum_{k=1}^m N_{ik}}$$
        where $N_{ij}$ is the observed count of transitions from state $i$ to state $j$.

        ### 3. Stationary Distribution ($\pi$)
        The long-run equilibrium vector $\boldsymbol{\pi} = [\pi_0, \pi_1, \pi_2]$
satisfies:
        $$\boldsymbol{\pi} P = \boldsymbol{\pi} \quad \text{subject to} \quad \sum_{i=1}^3
\pi_i = 1$$

        ### 4. Expected Sojourn (Holding) Time
        The expected number of consecutive days the market lingers in state $i$ before
switching:
        $$\mathbb{E}[T_i] = \sum_{k=1}^\infty k \cdot p_{ii}^{k-1}(1 - p_{ii}) = \frac{1}
{1 - p_{ii}}$$
        """)

# ----------------- TAB 3: REGIME-PARTITIONED PRICE ACTION -----------------
with tab3:
    st.plotly_chart(
        create_candlestick_regime_chart(df, ticker=ticker),
        use_container_width=True
    )

    st.markdown("#### 📋 Historical Regime Performance Breakdown")
    st.dataframe(pipeline["regime_stats"], use_container_width=True)

# ----------------- TAB 4: ML ARCHITECTURE & HOLD-OUT CHECK -----------------
with tab4:
    col_v1, col_v2 = st.columns([1, 1])
    with col_v1:
        st.markdown("#### ⚔️ Model Performance Benchmark")
        st.dataframe(eval_res["comparison_df"], use_container_width=True)
        st.caption("Scored on the last 30% of rows; models were fitted on the first 70%
only. "
                   "Regime labels come from a GMM fitted on the full history, so this is a
descriptive hold-out check, not a walk-forward test. "
                   "Daily returns are very noisy: expect R\u00b2 near zero and directional
accuracy near 50-55%.")

        st.markdown(r"""
        **Idea behind regime-conditioned regression (compare with the numbers above):**
        1. **Regime Heterogeneity:** A single global model assumes constant slope
coefficients $\boldsymbol{\beta}$ across calm and turbulent markets; separate models per
regime can fit each state differently.
        2. **Trade-off:** One model per regime also means fewer rows per model, which
raises estimation error. In-sample the regime models usually look better; on held-out data
the advantage often disappears.
```

```
        3. **Conditioned Residuals:** Shocks in the Monte Carlo are resampled from the active regime's in-sample residuals ($\sigma^2_{S_t}$), which captures regime-dependent volatility but understates out-of-sample error.
        """)

    with col_v2:
        st.plotly_chart(create_model_comparison_bar(eval_res["comparison_df"]), use_container_width=True)

# ----------------- TAB 5: FACULTY VIVA DEFENSE GUIDE -----------------
with tab5:
    st.markdown("""
    ### 🎓 Complete Viva Voce Defense Guide
    *Keep this tab handy during your project presentation or oral examination!*
    """)

    q1, q2 = st.columns(2)
    with q1:
        with st.container(border=True):
            st.markdown("##### Q1: Why not just use LSTM or Deep Learning?")
            st.markdown(r"""
            **Strong Answer:**
            LSTMs are notoriously prone to overfitting on financial time series due to the low signal-to-noise ratio. They behave as 'black boxes' and cannot quantify discrete market regime shifts.
            Our **Markov-ML hybrid model** delivers:
            - **Formal Mathematical Interpretability** (Transition Matrix $P$, stationary distributions $\pi$).
            - **Controlled overfitting risk** through regularised models (ridge, shallow forest); the hold-out check in Tab 4 shows how much, if any, edge survives.
            - **Direct probabilistic stress-testing** (simulating crashes by forcing regime transitions).
            """)

        with st.container(border=True):
            st.markdown("##### Q2: How do you address Non-Stationarity in Asset Prices?")
            st.markdown(r"""
            **Strong Answer:**
            Raw asset prices $P_t$ are non-stationary (unit root / $I(1)$ process). We transform prices into log-differenced returns $r_t = \ln(P_t/P_{t-1})$ which are covariance-stationary ($I(0)$).
            Furthermore, the **Markov Chain explicitly captures structural regime shifts** (volatility clustering), solving the classic regime-switching breakdown of classical econometric models.
            """)

    with q2:
        with st.container(border=True):
            st.markdown("##### Q3: How are the Markov states defined and trained?")
            st.markdown(r"""
            **Strong Answer:**
            We use **Gaussian Mixture Models (GMM)**, an unsupervised Expectation-Maximization (EM) algorithm, over multi-dimensional feature space (Daily Returns, 20-Day Realized Volatility, and Trend Spread).
            The clusters are then ranked by their average daily return (volatility is a clustering input, not the ranking rule):
            1. **Bull Regime:** Highest average return (typically with lower volatility).
            2. **Neutral Regime:** Middle average return, moderate consolidation.
            3. **Bear Regime:** Lowest average return (typically with elevated volatility).
            The GMM is fitted on the full history, so the regimes are a *descriptive* labelling of the past, not a real-time signal.
            """)

        with st.container(border=True):
            st.markdown("##### Q4: How does the Monte Carlo engine link with ML?")
            st.markdown(r"""
            **Strong Answer:**
            At each forward time-step $t+1$:
            1. The Markov transition row $P(S_{t+1} \mid S_t)$ determines the next market state.
            2. The ML model trained specifically on that regime predicts the drift $\mathbb{E}[r_{t+1}]$.
            3. A stochastic shock is sampled from that regime's empirical residual distribution $\epsilon \sim \text{Resid}(S_{t+1})$.
            This preserves volatility clustering and asymmetric downside tails without assuming a naive Gaussian distribution!
            """)

        with st.container(border=True):
            st.markdown("##### Q5: What are the limitations?")
```

```
            st.markdown(r"""
            **Strong Answer:**
            - The forward cones are *scenario simulations*, not out-of-sample price forecasts.
            - Regimes come from a GMM fitted on the whole history (look-ahead), and the number of regimes is fixed at three.
            - In the simulation only the return feature is updated each day; volatility, trend, RSI and momentum stay at their last observed values.
            - Residual shocks are in-sample, and 95% VaR is the 5th percentile of simulated horizon returns (noisy with few paths).
            """)

    st.markdown("---")
    st.markdown("#### 📄 Publication-Grade Academic Figure (Matplotlib 300-DPI)")
    st.caption("Generate and export a publication-quality 4-panel figure designed specifically for academic semester project reports, PowerPoint presentations, and LaTeX thesis submissions.")

    with st.expander("👁️ View & Export Matplotlib Academic Figure", expanded=False):
        with st.spinner("Rendering 300-DPI academic report figure..."):
            mpl_fig = generate_matplotlib_report_figure(
                df=df,
                transition_matrix=markov.transition_matrix,
                stationary_dist=markov.stationary_distribution,
                sim_results=sim_results,
                ticker=ticker
            )
            st.pyplot(mpl_fig)

            buf = io.BytesIO()
            mpl_fig.savefig(buf, format="png", dpi=300, bbox_inches="tight")
            st.download_button(
                label=f"📥 Download High-Res {ticker} Figure (300 DPI PNG)",
                data=buf.getvalue(),
                file_name=f"{ticker}_markov_ml_report_figure.png",
                mime="image/png"
            )
```